Chapter 15 — Interview Question Bank

Every HTML and CSS question that keeps showing up in front-end interviews, in one place: the classic h5bp list, the GreatFrontEnd and InterviewBit quizzes, popular CSS question guides, and the "build this layout" tasks from machine-coding rounds (timed rounds where you build a small UI live). Chapter 14 §14.6 already has 47 short answers; this bank includes all of them, links back to them, and adds the many questions that list does not have.

How to use this bank

- **Wording.** Each question is phrased the way interviewers usually phrase it, so you recognise it instantly.
- **★ = asked very often.** The question appears in several of the source lists *and* in reported interviews. If you are short on time, drill only the ★ questions (about a third of the bank).
- **Answers.** Each answer is a short version you can say out loud in 20–40 seconds: 2–6 sentences, or a small code snippet and a sentence or two. Terms are defined the first time they appear.
- **→ *More:*** links take you to the full explanation (with real browser output) in this book. A question with no link is not covered elsewhere in the book, so its answer here is complete on its own. "§14.6 Qn" means the question is also in Chapter 14's short list.
- **How to drill.** Cover the answer, say yours out loud (as if to an interviewer), then uncover and compare. Mark the ones you missed, follow the link, and come back two days later. For the "build this" questions, type the code from memory into a blank `.html` file and open it in a browser.
- **Versions.** Browser-support statements match the rest of the book (checked September 2026, current browsers Chrome 154, Safari 26, Firefox 15x). "Baseline" is the label for a feature that works in all major browsers (Chrome, Edge, Firefox, Safari); "newly available" means it only recently reached the last of them.

HTML documents, the `<head>` and elements

Q1. ★ What does a doctype do?

`<!doctype html>` is the first line of the file. It tells the browser to render the page in **standards mode** (follow the current CSS specifications). Without it, browsers fall back to **quirks mode**, which imitates old browser bugs — for example, tables do not inherit the page's font size, percentage heights and `line-height` are calculated differently, and sloppy CSS such as hex colours without `#` or lengths without units is accepted. (The old IE box-model quirk — `width` including padding and border — survives in modern browsers only for a few special cases such as table cells' height.) It is not an HTML element and is case-insensitive. HTML5's doctype is deliberately short because it exists only to trigger standards mode.

Follow-up: Can anything come before the doctype? Only a byte-order mark (BOM, an invisible marker some editors add) and comments/whitespace are tolerated; a stray visible character (any text) above it flips the page into quirks mode, and in old Internet Explorer even a comment or XML prolog before it did.

→ *More:* §2.2 Document structure and the doctype, §14.6 Q1

Q2. What are the different kinds of doctypes, and which one should you use?

HTML 4.01 and XHTML 1.0 had Strict, Transitional and Frameset doctypes that pointed to a DTD (Document Type Definition, a formal grammar file). Some of the long ones trigger **almost-standards mode** (standards mode except that images in table cells keep the old line-height gap). Today you always write `<!doctype html>`: it gives full standards mode in every browser.

→ *More:* §2.2 Document structure and the doctype

Q3. What is the difference between standards mode and quirks mode — how can you tell which one a page is in?

Standards mode follows the specs; quirks mode emulates old browser behaviour (box sizing, table and inline layout differences). Run `document.compatMode` in the console: `"CSS1Compat"` is standards mode, `"BackCompat"` is quirks mode. A missing or malformed doctype, or anything before it, can drop the page into quirks mode.

→ *More:* §2.2 Document structure and the doctype

Q4. What is the purpose of the `<head>` and `<body>` elements?

`<head>` holds **metadata** — information about the page that is not shown in it: the character encoding, `<title>`, viewport settings, links to stylesheets, scripts, icons, and SEO/social tags. `<body>` holds everything the user sees and interacts with. The browser creates both even if you leave the tags out, but writing them makes the structure clear.

→ *More:* §2.2 Document structure and the doctype, §2.3 The `<head>`: metadata that matters

Q5. ⋆ Why should `<meta charset="utf-8">` be the first thing in `<head>`?

The browser must know the **character encoding** (how bytes map to characters) before it decodes the text. The declaration has to appear within the first 1024 bytes; if it comes later, the browser may guess, and characters like "é" or "₹" can turn into garbage (mojibake), or the parser may have to restart. UTF-8 covers every language and is the only encoding you should use.

Follow-up: What if the server's `Content-Type` header says a different charset? The HTTP header wins over the `<meta>` tag, so fix the server configuration as well.

→ *More:* §2.3 The `<head>`: metadata that matters, §14.6 Q2

Q6. ⋆ Which tags belong in a minimal, production-ready `<head>`?

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Page title – Site</title>
<meta name="description" content="One-sentence summary for search results">
<link rel="canonical" href="https://example.com/page">
<link rel="icon" href="/favicon.svg">
<link rel="stylesheet" href="/app.css">
<script src="/app.js" defer></script>
```

Plus `lang` on `<html>`, and optionally Open Graph tags (`og:title`, `og:image`) for link previews.

→ *More:* §2.3 The `<head>`: metadata that matters, §14.7 Cheat sheets

Q7. What does `<meta name="viewport" content="width=device-width, initial-scale=1">` do?

Mobile browsers otherwise lay a page out on a virtual screen about 980 px wide and shrink it, so your media queries never fire and text is tiny. This tag sets the layout width to the device's width in CSS pixels and the starting zoom to 100%. Never add `user-scalable=no` or `maximum-scale=1`: blocking zoom fails WCAG.

→ *More:* §10.2 The viewport meta tag and viewport units, §14.6 Q34

Q8. ⋆ What are void (empty) elements in HTML?

Elements that cannot have content and so have no closing tag: `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`, `<source>`, `<track>`, `<wbr>`, `<area>`, `<base>`, `<col>`, `<embed>`. The trailing slash in `<br />` is allowed but ignored in HTML. Writing `<div />` does **not** close a div — the parser treats it as an opening tag.

Follow-up: Can a custom element be void? No — `<my-icon />` is parsed as an opening tag only; custom elements always need a closing tag.

→ *More:* §2.1 Elements, tags and attributes

Q9. Are HTML tags and elements the same thing?

No. A **tag** is the text markup (`<p>`, `</p>`); an **element** is the whole thing — start tag, content, end tag — and becomes a node (an object) in the DOM (Document Object Model, the in-memory tree the browser builds from the HTML). Some elements have optional tags (the browser inserts `<tbody>` or `</li>` for you), so an element can exist without its tags being written.

→ *More:* §2.1 Elements, tags and attributes

Q10. ⋆ What are `data-*` attributes good for, and when should you avoid them?

They store custom data on an element that has no standard attribute: `<li data-id="42" data-status="done">`. JavaScript reads them via `el.dataset.id` (dashes become camelCase: `data-user-id` → `dataset.userId`), and CSS can select on them (`[data-status="done"]`). Avoid them for anything that has a real attribute or ARIA state (use `disabled`, `aria-expanded`), for large data (keep it in JS), and for secrets — they are visible in the page source.

Follow-up: How do you style based on a data attribute? `[data-state="open"] .panel { display: block; }` — but for states that assistive technology should know about, prefer the ARIA attribute (`[aria-expanded="true"]`) and style that, so the visual and announced states can't drift apart.

→ *More:* §2.1 Elements, tags and attributes

Q11. What is the difference between the `id` and `class` attributes?

`id` must be unique in the document; it is a target for fragment links (`#section`), `<label for>`, ARIA references and `getElementById`. `class` can be shared by many elements and an element can have many classes; it is the normal hook for CSS. In CSS an ID selector has much higher specificity than a class, which is one reason style guides avoid IDs for styling.

→ *More:* §2.1 Elements, tags and attributes, §6.4 Specificity

Q12. What are global attributes? Name a few.

Attributes allowed on every element: `id`, `class`, `style`, `title`, `lang`, `dir`, `hidden`, `tabindex`, `data-*`, `contenteditable`, `draggable`, `inert`, `popover`, `translate`, `spellcheck`, `accesskey`, plus all ARIA `role`/`aria-*` attributes and event handler attributes (`onclick`).

→ *More:* §2.1 Elements, tags and attributes

Q13. ⋆ `<b>` vs `<strong>`, `<i>` vs `<em>`?

`<strong>` means strong importance or urgency; `<em>` means stress emphasis that changes the sentence's meaning ("I *never* said that"). `<b>` and `<i>` carry no importance: `<b>` draws attention (keywords, product names), `<i>` marks an alternate voice (foreign words, technical terms, ship names). All four look bold/italic by default — choose by meaning, then style with CSS.

Follow-up: Is `<b>` deprecated? No — it was redefined in HTML5 with a specific meaning (drawing attention without extra importance) and is still valid.

→ *More:* §2.6 Text-level semantics

Q14. What are character references (HTML entities), and when do you need them?

A way to write characters that would otherwise be parsed as markup or are hard to type: `<` (<), `>` (>), `&` (&), `"` ("), ` ` (non-breaking space), or a numeric code like `₹` (₹). With UTF-8 you only *need* them for `<` and `&` in text and for quotes inside attribute values.

→ *More:* §2.9 Character references and comments

Q15. ⋆ What is the difference between `<script>`, `<script async>` and `<script defer>`?

- Plain `<script src>`: HTML parsing **stops** while the file downloads and runs.
- `async`: downloads in parallel and runs **as soon as it arrives**, pausing parsing then; order between async scripts is not guaranteed. Good for independent scripts like analytics.
- `defer`: downloads in parallel and runs **after parsing finishes**, before `DOMContentLoaded`, **in document order**. Best default for your app code. Module scripts (`type="module"`) are deferred by default. `async`/`defer` do nothing on inline scripts (except `async` on modules).

Follow-up: Which one runs first, a `defer` script or `DOMContentLoaded` handlers? Deferred scripts run first, then `DOMContentLoaded` fires. `async` scripts may run before or after `DOMContentLoaded`.

→ *More:* §14.2 Render-blocking resources, §14.6 Q44

Q16. ⋆ Why is it a good idea to put CSS `<link>`s in the `<head>` and JS `<script>`s just before `</body>`? Any exceptions?

CSS in the head lets the browser fetch it early and paint the page styled once, avoiding a **flash of unstyled content** (FOUC). Scripts at the end used to stop them from blocking parsing of the content above. Today the better answer is `<script defer>` in the head: it downloads early *and* does not block

parsing. Exceptions: tiny scripts that must run before first paint (the dark-mode theme script, feature detection) go inline in the head; async analytics can go anywhere.

Follow-up: Does a stylesheet block JavaScript too? Yes: a synchronous script that comes after a stylesheet waits for that stylesheet, because the script might read computed styles.

→ *More:* §14.2 Render-blocking resources, §13.14 Theming and dark mode with custom properties

Q17. What is progressive rendering?

Showing useful content as early as possible instead of waiting for the whole page: streaming HTML so the browser parses and paints the top first, inlining critical CSS, deferring scripts, lazy-loading below-the-fold images, and (in apps) server-side rendering with streaming. The aim is a fast First Contentful Paint and Largest Contentful Paint.

→ *More:* §14.1 The critical rendering path, §14.4 Fixing each metric (and the other basics)

Q18. What are `preload`, `prefetch`, `preconnect` and `dns-prefetch`, and when do you use each?

They are `<link rel>` **resource hints**:

- `preload`: fetch a resource *this page* needs soon at high priority (a hero image, a font discovered late in CSS). Needs `as="..."`.
- `prefetch`: low-priority fetch of something the *next* navigation will probably need.
- `preconnect`: open the DNS + TCP + TLS connection to another origin early (a font CDN).
- `dns-prefetch`: only the DNS lookup; a cheap fallback. Overusing `preload` steals bandwidth from more important files.

→ *More:* §11.2 Loading fonts well: `font-display`, preload, subsetting, §14.4 Fixing each metric (and the other basics)

Q19. What do `rel="noopener"` and `rel="noreferrer"` do, and why do they matter for `target="_blank"`?

A page opened in a new tab used to get `window.opener`, a handle to your page, and could redirect it to a phishing copy ("reverse tabnabbing"). `noopener` removes that handle; `noreferrer` also hides the `Referer` header (and implies `noopener`). Modern browsers apply `noopener` to `target="_blank"` links by default, but writing it is still common practice.

→ *More:* §2.8 Links

Q20. `<link>` vs `<a>`?

`<a href>` is a hyperlink the user clicks to navigate; it goes in the body. `<link>` is metadata in the `<head>` that relates the document to another resource — a stylesheet, an icon, a canonical URL, a preload — and is never clicked.

→ *More:* §2.3 The `<head>`: metadata that matters, §2.8 Links

Q21. What is the `<base>` element, and what problems can it cause?

`<base href="https://cdn.example.com/">` changes the base URL for *every* relative URL in the page, and `<base target>` sets the default link target. It surprises people: in-page links like `href="#top"` resolve against the base and navigate away, and single-page-app routers can break. There can be only one, and it must come before any URL in the head. Use it rarely.

Q22. What is Subresource Integrity (SRI)?

An `integrity` attribute containing a hash of the expected file: `<script src="https://cdn…/lib.js" integrity="sha384-…" crossorigin="anonymous">`. If the CDN (Content Delivery Network, servers that host files close to users) serves a modified file, the hash does not match and the browser refuses to run it. It protects you from a compromised third-party host.

Q23. How do you serve a page with content in multiple languages?

Set the page language on `<html lang="en">` and mark passages in another language with `lang` on that element (`<span lang="fr">`). Serve one URL per language (`/en/`, `/fr/`), list alternates with `<link rel="alternate" hreflang="fr" href="…">`, and let users switch language explicitly — use `Accept-Language` only as a first guess. Screen readers use `lang` to switch pronunciation; browsers use it for hyphenation, fonts and translation.

→ *More:* §2.2 Document structure and the doctype, §14.5 SEO basics

Q24. What must you be careful about when designing for multilingual sites?

Text length varies (German can be 30% longer than English), so avoid fixed-width buttons; right-to-left languages (Arabic, Hebrew) need `dir="rtl"` and logical CSS properties (`margin-inline-start` instead of `margin-left`); don't put text inside images; don't build sentences by concatenating strings (word order differs); date, number and currency formats differ; fonts must cover the scripts; and colours or icons can carry different meanings.

→ *More:* §13.6 Logical properties

Q25. What are the building blocks of "HTML5" as an open web platform?

"HTML5" became shorthand for the modern platform: semantic elements (`<header>`, `<nav>`, `<main>`, `<article>`), native media (`<video>`, `<audio>`), graphics (`<canvas>`, inline SVG), new form input types and built-in validation, storage (`localStorage`, IndexedDB), offline (service workers), real-time communication (WebSockets, server-sent events), Web Workers, and device APIs (geolocation). HTML itself is now a "Living Standard" maintained by WHATWG, with no version numbers.

→ *More:* §1.1 The web in one picture

Q26. Have you used HTML templating languages? What are they for?

Templating languages (Handlebars, Nunjucks, EJS, Pug, Jinja, Thymeleaf, JSX in React) generate HTML from data plus a template with loops, conditions and partials (reusable fragments). Good answers mention that they must **escape** output by default (turn `<` into `<`) to prevent XSS — cross-site scripting, where an attacker's text gets run as script. The native `<template>` element is different: it holds inert HTML that JavaScript clones at runtime.

Q27. How do you validate HTML, and why bother?

Run it through the W3C Nu HTML Checker (validator.w3.org/nu) or an editor/CI linter (html-validate). Browsers silently fix invalid markup, but each one may fix it differently, and invalid nesting (a `<div>` inside a `<p>`, a button inside a link) often breaks layout, scripts or accessibility.

→ *More:* §2.10 Validation

Semantic HTML and SEO

Q28. ⋆ What is semantic HTML, and why does it matter?

Using elements for their meaning — `<nav>` for navigation, `<button>` for actions, `<h2>` for a section heading — rather than generic `<div>`s styled to look right. It matters because browsers give semantic elements built-in behaviour (keyboard support, focus), assistive technology (software such as screen readers) relies on the meaning, search engines understand the page better, and the code is easier to read.

Follow-up: Give an example where semantics changed behaviour for you: e.g. replacing a clickable `div` with `<button>` fixed keyboard access, or using `<nav>` and `<main>` let screen-reader users jump between regions.

→ *More:* §3.1 Why semantics and accessibility, §3.2 Landmarks

Q29. ⋆ `<section>` vs `<article>` vs `<div>`?

`<article>` is self-contained content that would make sense on its own (a blog post, a product card, a comment). `<section>` is a thematic group of content that normally has a heading. `<div>` has no meaning; use it purely as a styling/layout wrapper.

Follow-up: Can an `<article>` contain `<section>`s, and vice versa? Yes, both ways: a long article split into sections, or a section listing several articles (blog teasers).

→ *More:* §3.2 Landmarks, §14.6 Q4

Q30. ⋆ What are landmarks, and why do they matter?

Landmarks are page regions that assistive technology lists so users can jump straight to them: `<header>` (banner, when not inside article/section), `<nav>` (navigation), `<main>` (main), `<aside>` (complementary), `<footer>` (contentinfo), `<search>`, and `<form>`/`<section>` when they have an accessible name. Have exactly one visible `<main>`.

Follow-up: Do you still need `role="main"` on `<main>`? No — every modern browser maps the element to the role; adding the role is redundant.

→ *More:* §3.2 Landmarks, §14.6 Q5

Q31. What is the relationship between `<header>` and `<h1>`? Can a page have several `<header>`s or `<h1>`s?

`<header>` is a container for introductory content (logo, nav, a heading); `<h1>` is a heading. A page can have many `<header>`s (one per article or section). Several `<h1>`s are valid, but the "document outline" algorithm that would rank them by nesting was never implemented by browsers, so use one `<h1>` per page and real levels (`h2`, `h3`) for structure.

→ *More:* §2.4 Headings, §3.3 Headings and the outline

Q32. Why must headings not skip levels, and why should you not pick a heading for its size?

Screen-reader users navigate by heading level to understand the page structure; jumping from `<h2>` to `<h4>` suggests missing content. Choose the level by structure, then set the size with CSS.

→ *More:* §2.4 Headings, §3.3 Headings and the outline

Q33. `<figure>` vs `<img>`?

`<img>` is an image. `<figure>` is a self-contained unit referenced from the main text — an image, chart, code listing or quote — optionally with a `<figcaption>`. The caption is visible to everyone; `alt` is the replacement text when the image cannot be seen. They do different jobs, so a figure image still needs `alt`.

→ *More:* §4.5 `<figure>` and `<figcaption>`

Q34. ⋆ Which HTML matters for SEO?

SEO (Search Engine Optimisation) is making pages easy for search engines to find, understand and rank. The HTML that matters: a unique `<title>` and `<meta name="description">`, `<link rel="canonical">`, `lang`, the viewport tag, no stray `noindex`, one clear `<h1>` and logical headings, real `<a href>` links (crawlers don't click buttons), descriptive `alt` text, semantic structure, and structured data (JSON-LD). Speed (Core Web Vitals) also counts.

Follow-up: Does the `keywords` meta tag help SEO? No — Google has ignored it for many years. Content quality, links and page experience matter.

→ *More:* §14.5 SEO basics, §14.6 Q47

Q35. What is a canonical URL?

`<link rel="canonical" href="…">` tells search engines which URL is the main version when the same content is reachable at several URLs (with tracking parameters, `http` vs `https`, `/page` vs `/page/`). It is a strong hint, not a command; it consolidates ranking signals onto one URL.

→ *More:* §14.5 SEO basics

Q36. `robots.txt` `Disallow` vs `<meta name="robots" content="noindex">`?

`robots.txt` controls **crawling** (fetching). `noindex` controls **indexing** (showing in results). A disallowed URL can still appear in results if other sites link to it — and Google can't see a `noindex` on a page it isn't allowed to fetch. To keep a page out of results, allow crawling and use `noindex`.

→ *More:* §14.5 SEO basics

Q37. What is structured data (JSON-LD, microdata), and why use it?

Machine-readable facts about the page using the schema.org vocabulary — product price and rating, article author and date, breadcrumbs, FAQs. Search engines can then show **rich results** (stars, prices). Google recommends JSON-LD: a `<script type="application/ld+json">` block, separate from the visible markup. **Microdata** (`itemscope`, `itemprop` attributes woven into the HTML) is the older alternative.

→ *More:* §14.5 SEO basics

Q38. How do client-side rendered single-page apps affect SEO, and what do you do about it?

In a **single-page app** (SPA) the server sends an almost-empty HTML shell and JavaScript builds the page. Google can run JavaScript, but rendering is delayed and other crawlers and link-preview bots often don't run it at all. Fixes: server-side rendering (SSR) or static generation so the HTML arrives complete, real `<a href>` links, unique titles and canonicals per route, and correct status codes (a real 404, not a 200 "not found" page).

→ *More:* §14.5 SEO basics

Accessibility (ARIA, keyboard and focus, WCAG)

Q39. What is web accessibility, and who benefits?

Accessibility (a11y) means people with disabilities can perceive, operate and understand the site: blind users with screen readers (software that reads the page aloud), people who use only a keyboard or switch devices, users with low vision who zoom to 400%, colour-blind users, deaf users who need captions, and people with cognitive or vestibular (motion-sensitivity) conditions. Everyone else benefits too — captions in a noisy train, keyboard shortcuts, clear forms — and in many countries it is a legal requirement.

→ *More:* §3.1 Why semantics and accessibility

Q40. ⋆ What is ARIA, and what is the first rule of ARIA?

ARIA (Accessible Rich Internet Applications) is a set of attributes — `role`, `aria-*` states and properties — that tell assistive technology what a custom widget is (`role="tab"`), what state it is in (`aria-expanded="true"`) and how it relates to others (`aria-controls`). It changes only what is *announced*, not behaviour: it adds no keyboard support or styling. First rule: if a native element does the job (`<button>`, `<input type="checkbox">`, `<dialog>`), use it instead. No ARIA is better than bad ARIA.

Follow-up: Name a common ARIA mistake. Adding `role="button"` to a `div` without making it focusable or keyboard-operable; `aria-hidden="true"` on a focusable element (it can be tabbed to but is announced as nothing); or `aria-label` on a `div`/`span` with no role, where most screen readers ignore it.

→ *More:* §3.6 ARIA, §14.6 Q6

Q41. What is the accessibility tree?

A parallel tree the browser builds from the DOM for assistive technology. Each node has a **role** (button, link, heading), an **accessible name** (what is read out), a **state** (checked, expanded, disabled) and a description. Elements hidden with `display: none`, `hidden` or `aria-hidden="true"` are left out. You can inspect it in DevTools' Accessibility pane.

→ *More:* §3.4 The accessibility tree

Q42. ⋆ `aria-label` vs `aria-labelledby` vs `aria-describedby` vs `<label>` — how is the accessible name computed?

In priority order: `aria-labelledby` (points to other elements' IDs, their text becomes the name) → `aria-label` (a string on the element) → native labelling (`<label for>`, `alt`, `<caption>`, `<legend>`) → the element's text content (for buttons, links) → `title` as a last resort. `aria-describedby` adds a secondary **description** (hints, error messages) read after the name. Prefer visible text and `<label>` so sighted and non-sighted users get the same words.

Follow-up: What does `aria-label` on a link with visible text do? It replaces the visible text as the accessible name, so voice-control users saying the visible words may not be able to activate it (WCAG 2.5.3 Label in Name). Keep the visible text at the start of the accessible name.

→ *More:* §3.4 The accessibility tree, §3.8 Accessible forms

Q43. How do you make a custom component (for example, a `<div>` "button") accessible — and why is `<button>` better?

A `div` with `onclick` is not focusable, not announced as a button and doesn't react to Enter/Space. To fix it you would need `role="button"`, `tabindex="0"`, key handlers for Enter and Space, a disabled state and focus styles. `<button type="button">` gives all of that for free, which is why the answer is "use the native element".

→ *More:* §3.7 Keyboard and focus

Q44. ⋆ `<button>` vs `<a>` — when do you use which?

Links **navigate** (they have an `href`, open in a new tab, show in history, are activated by Enter). Buttons **perform an action** on the page (submit, open a menu, toggle). A link that looks like a button is

fine; a button used for navigation, or an `<a>` without `href` used as a button, breaks keyboard behaviour and confuses screen-reader users.

Follow-up: What about `<a href="#" onclick>`? It works with a mouse but announces as a link, adds a history entry or jumps to the top, and isn't activated by Space — use a `<button>`.

→ *More:* §2.8 Links, §14.6 Q8

Q45. How does `tabindex` work? Why avoid positive values?

`tabindex="0"` adds an element to the natural Tab order (at its DOM position); `tabindex="-1"` makes it focusable by script (`el.focus()`) but not by Tab — useful for headings you move focus to, or dialog containers. Positive values (`tabindex="5"`) jump the element ahead of everything else, creating a confusing order that is hard to maintain — avoid them.

→ *More:* §3.7 Keyboard and focus

Q46. ⋆ How do you manage focus in a modal dialog?

On open: move focus into the dialog (the first control or the dialog itself); keep Tab inside it (a **focus trap**); make the page behind it inert (unreachable). Escape closes it. On close: return focus to the button that opened it. Native `<dialog>` opened with `showModal()` does almost all of this: the rest of the page becomes inert, Escape closes, and it renders in the top layer.

```
<button onclick="dlg.showModal()">Edit</button>
<dialog id="dlg" aria-labelledby="t"><h2 id="t">Edit profile</h2>
  <form method="dialog"><button>Close</button></form></dialog>
```

Follow-up: Why is `aria-modal="true"` not enough? It only tells screen readers the rest is inert; it doesn't trap keyboard focus or block clicks. `showModal()` or the `inert` attribute on the background actually does.

→ *More:* §3.7 Keyboard and focus

Q47. ⋆ What is a visible focus indicator, and why should you never write `outline: none` on its own?

The ring around the focused control shows keyboard users where they are; WCAG requires it to be visible. Removing it without a replacement makes the site unusable by keyboard. Use `:focus-visible`, which shows the ring for keyboard focus but not for mouse clicks:

```
:focus-visible { outline: 3px solid #1d4ed8; outline-offset: 2px; }
```

→ *More:* §3.7 Keyboard and focus, §11.6 Borders, radius and outlines

Q48. What is a skip link?

The first focusable element on the page — "Skip to main content", linking to `#main` — usually visually hidden until it receives focus. It lets keyboard and screen-reader users jump past the repeated header and navigation on every page.

→ *More:* §3.7 Keyboard and focus

Q49. ⋆ What are the ways to hide content, and what does each hide it from?

Technique	Visible?	Screen reader?	Takes space?
`display: none` / `hidden` attribute	No	No	No
`visibility: hidden`	No	No	Yes
`opacity: 0`	No	**Yes** (still focusable)	Yes
`aria-hidden="true"`	Yes	No	Yes
`.visually-hidden` class	No	**Yes**	No (1px)
`inert`	Yes	No (and unfocusable)	Yes

Follow-up: Which one would you use for a closed dropdown panel? `hidden` / `display: none`, so its links are not focusable while closed. For an animated panel, `visibility: hidden` after the transition (or `inert`) keeps focus out.

→ *More:* §3.4 The accessibility tree, §3.6 ARIA

Q50. ⋆ How do you hide content visually but keep it available to screen readers?

A "visually hidden" (or `sr-only`) utility clips the element to 1×1 px, so it is not seen but still in the accessibility tree:

```
.visually-hidden {
  position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px;
  overflow: hidden; clip-path: inset(50%); white-space: nowrap; border: 0;
}
```

Use it for icon-button labels, skip links (until focused) and extra table context. Don't use `display: none` — that hides it from everyone.

→ *More:* §3.6 ARIA

Q51. ⋆ How do you write good alt text?

Describe the image's *purpose in this context*, not its pixels: a logo link gets the destination ("Acme home"), a chart gets its key message, a photo in an article gets what the reader needs to know. Decorative images get `alt=""` (empty, so screen readers skip them). Never omit `alt` — the screen reader may read the file name. Don't start with "image of".

Follow-up: What is the alt for an image that is a link with no other text? The link's destination or purpose ("Search", "Home"), because it becomes the link's accessible name.

→ *More:* §3.5 Alt text, §4.1 The `<img>` element, §14.6 Q7

Q52. What is WCAG? What do the levels A, AA and AAA mean?

WCAG (Web Content Accessibility Guidelines, currently 2.2) is the W3C's standard, organised under four principles — **POUR**: Perceivable, Operable, Understandable, Robust. Each success criterion has a level: A (minimum), AA (the usual legal and contractual target), AAA (enhanced, not required for whole sites).

→ *More:* §3.9 WCAG 2.2

Q53. ⋆ What contrast ratio does WCAG AA require?

4.5:1 for normal text, 3:1 for large text (≥ 24 px, or ≥ 18.66 px bold), and 3:1 for UI components and meaningful graphics (borders of inputs, focus rings, icons). AAA raises text to 7:1 and 4.5:1. The ratio is computed from the relative luminance of the two colours: (L1 + 0.05) / (L2 + 0.05).

Follow-up: Does contrast apply to placeholder text and disabled controls? Placeholder text must meet 4.5:1 if it conveys information; disabled (inactive) controls are exempt.

→ *More:* §3.9 WCAG 2.2, §11.4 Colour and contrast, §14.6 Q13

Q54. What is new in WCAG 2.2?

Nine new criteria (six at level A/AA), including **Focus Not Obscured (Minimum)**, AA (a sticky header must not fully cover the focused element), **Target Size (Minimum)**, AA (pointer targets at least 24×24 CSS px, or enough spacing that a 24 px circle around each doesn't overlap another), **Dragging Movements** (offer a non-drag alternative), **Consistent Help**, **Redundant Entry** (don't ask for the same info twice) and **Accessible Authentication (Minimum)**, AA (no memory or puzzle tests like retyping a code without allowing paste or password managers); the other three are AAA. 4.1.1 Parsing was removed.

→ *More:* §3.9 WCAG 2.2

Q55. What is an ARIA live region?

An element whose changes screen readers announce without moving focus — for toasts, "3 results found", form-save confirmations. `aria-live="polite"` (or `role="status"`) waits until the user is idle; `aria-live="assertive"` (or `role="alert"`) interrupts. The region must already exist in the DOM *before* you change its text, or the change may not be announced.

→ *More:* §3.6 ARIA, §5.7 Accessible error messages

Q56. How do you build an accessible accordion, tabs or dropdown menu?

Accordion: each header is a `<button aria-expanded aria-controls>` inside a heading (or use native `<details>/<summary>`). Tabs: `role="tablist"`, `role="tab"` with `aria-selected`, `role="tabpanel"`, arrow keys move between tabs. Disclosure menu: a `<button aria-expanded>` that toggles a list of links. Follow the

WAI-ARIA Authoring Practices Guide (APG) keyboard patterns — and prefer native elements where they exist.

→ *More:* §3.6 ARIA, §10.4 Mobile-first in practice

Q57. How do you test accessibility?

Three layers: **automated** tools (axe DevTools, Lighthouse, WAVE) catch roughly a third of issues — missing alt, contrast, labels; a **manual keyboard test** (Tab through everything: can you reach, see and operate every control? any traps?); and a **screen-reader pass** (VoiceOver, NVDA) plus zoom to 200–400% and reduced motion. Real users with disabilities are the final test.

→ *More:* §3.10 Testing tools

Q58. What is `prefers-reduced-motion`, and why respect it?

A media feature that is `reduce` when the user has asked the OS to minimise animation. Large motion (parallax, zooms, auto-playing carousels) can cause nausea or dizziness for people with vestibular disorders. Wrap non-essential motion in `@media (prefers-reduced-motion: no-preference) { … }` or tone it down under `reduce`.

→ *More:* §12.9 Reduced motion and accessibility, §14.6 Q40

Forms and validation

Q59. How does a form submit? What do `action`, `method` and `name` do?

On submit the browser collects every enabled control that has a `name`, builds `name=value` pairs, and sends them to the `action` URL using `method` (GET or POST). A field without a `name` is **not sent**. `enctype` sets the encoding: `application/x-www-form-urlencoded` (default) or `multipart/form-data` (required for file uploads).

→ *More:* §5.2 How a form works — `action`, `method`, `name`

Q60. ⋆ GET vs POST in forms?

GET puts the data in the URL query string — bookmarkable, cacheable, shows in history and server logs; use it for searches and filters. POST sends the data in the request body — use it for anything that changes data, for passwords and other sensitive data, and for large or file uploads.

Follow-up: Is POST more secure than GET? Only slightly: the data isn't in the URL, logs or history, but it's still readable on the wire unless you use HTTPS, which is what actually protects it.

→ *More:* §5.2 How a form works — `action`, `method`, `name`, §14.6 Q10

Q61. ⋆ How do you label form fields accessibly?

Every input needs a `<label for="id">` (or the input wrapped inside the label) — clicking the label focuses the input, and it becomes the accessible name. Placeholders are not labels (they disappear and have low contrast). Group related radios or checkboxes in `<fieldset>` with a `<legend>`. Link hints and errors with `aria-describedby`.

Follow-up: Is `aria-label` an acceptable substitute for `<label>`? Only when there truly is no visible label (a search field with a visible magnifying-glass button, for example). A visible label helps everyone, including people with cognitive disabilities.

→ *More:* §3.8 Accessible forms, §14.6 Q9

Q62. What do `<fieldset>` and `<legend>` do?

`<fieldset>` groups related controls; `<legend>` is its caption, read before each control's label ("Delivery speed, Express, radio button"). A disabled fieldset disables every control inside it.

→ *More:* §5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

Q63. Name the HTML5 input types, and why use them instead of `type="text"`.

`email`, `url`, `tel`, `number`, `range`, `date`, `time`, `datetime-local`, `month`, `week`, `color`, `search`, `password`, `file`, `checkbox`, `radio`, `hidden`. They give the right mobile keyboard, built-in validation (`email` checks the format) and native pickers. `inputmode` changes only the keyboard (`inputmode="numeric"` for a PIN, which shouldn't be `type="number"`).

→ *More:* §5.3 Input types

Q64. ⋆ How does built-in (constraint) validation work?

Attributes declare rules — `required`, `type="email"`, `min` / `max`, `minlength` / `maxlength`, `pattern`, `step` — and the browser blocks submission and shows a message when one fails. In JavaScript, `input.validity` gives flags (`valueMissing`, `typeMismatch`, `patternMismatch` …), `checkValidity()` / `reportValidity()` test them, and `setCustomValidity("message")` adds your own error (pass `""` to clear it). `novalidate` on the form turns the browser UI off so you can show your own messages while keeping the API.

Follow-up: Why might `pattern` not work on `type="number"`? `pattern` only applies to text-like inputs (`text`, `email`, `tel`, `url`, `search`, `password`). For numbers use `min`, `max` and `step`.

→ *More:* §5.5 Built-in validation and the Constraint Validation API

Q65. Is client-side validation enough?

No. It is a convenience for the user; anyone can bypass it (DevTools, curl, disabling JS). The server must validate and sanitise everything again — client-side checks are UX, server-side checks are security.

→ *More:* §5.2 How a form works — `action`, `method`, `name`, §5.5 Built-in validation and the Constraint Validation API

Q66. `:invalid` vs `:user-invalid`?

`:invalid` matches as soon as the value breaks a rule — so a `required` field is red before the user has typed anything. `:user-invalid` (Baseline since 2023) matches only after the user has interacted with the field or tried to submit, which is the behaviour you normally want for error styling.

→ *More:* §5.5 Built-in validation and the Constraint Validation API

Q67. `readonly` vs `disabled`?

`readonly`: the user can focus and select the value but not change it, and the value **is submitted**. `disabled`: not focusable, greyed out, and the value is **not submitted**; it also skips validation. Use `aria-disabled="true"` when you want a control to stay focusable (so screen-reader users can discover it) but not work.

→ *More:* §5.3 Input types

Q68. `<select>` vs `<datalist>`?

`<select>` forces a choice from a fixed list. `<datalist>` attached to an `<input list="id">` offers suggestions while still allowing free text (an autocomplete). Use radios instead of a select when there are only a few options — all are visible at once.

→ *More:* §5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

Q69. `<progress>` vs `<meter>`; what is `<output>`?

`<progress>` shows completion of a task (upload 40%), and has an indeterminate state with no value. `<meter>` shows a scalar measurement within a known range (disk 80% full, password strength), with `low` / `high` / `optimum` thresholds. `<output>` holds the result of a calculation or user action and is announced as a live region in most screen readers.

→ *More:* §5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

Q70. What does the `autocomplete` attribute do?

It tells the browser and password manager what a field contains — `autocomplete="email"`, `"given-name"`, `"street-address"`, `"one-time-code"`, `"current-password"`, `"new-password"`. Correct values let users fill forms in one tap and are required by WCAG 1.3.5 (Identify Input Purpose).

→ *More:* §5.6 `autocomplete` — help the browser fill the form

Q71. How do you show accessible error messages?

Put the error text next to the field, link it with `aria-describedby`, set `aria-invalid="true"`, and on submit move focus to the first invalid field (or to an error summary at the top that links to each field). Don't rely on colour alone — add an icon or text. Say what went wrong and how to fix it.

→ *More:* §5.7 Accessible error messages

Q72. What is the difference between `<button>`, `<button type="button">` and `<input type="submit">`?

A `<button>` inside a form defaults to `type="submit"`, so a "Show password" button without `type="button"` submits the form — a classic bug. `<input type="submit">` can only contain plain text (its `value`); `<button>` can contain HTML such as an icon. `type="reset"` clears the form and is rarely wanted.

→ *More:* §5.2 How a form works — `action`, `method`, `name`

Images, media and embeds

Q73. ⋆ How do `srcset` and `sizes` work? Explain how the browser picks an image.

`srcset` lists candidate files with their intrinsic widths (`hero-800.jpg 800w, hero-1600.jpg 1600w`). `sizes` tells the browser how wide the image will be displayed (`(min-width: 60rem) 50vw, 100vw`), because it has to choose *before* CSS is loaded. The browser takes the first matching `sizes` entry, multiplies by the device pixel ratio (e.g. 400 px × 2 = 800 px needed), and picks the smallest candidate that is wide enough (it may also consider cached files or data-saver mode). With only density descriptors (`1x, 2x`) no `sizes` is needed.

Follow-up: Why can't the browser just use CSS to decide? Image requests start during HTML parsing (by the preload scanner, which looks ahead in the HTML for files to fetch) before layout is known — `sizes` gives it the layout hint up front. `sizes="auto"` (Chromium and Firefox so far, not Safari — not Baseline) works for lazy-loaded images, whose layout is known by the time they are fetched.

→ *More:* §4.3 Responsive images — `srcset`, `sizes` and `<picture>`, §14.6 Q11

Q74. ⋆ When do you use `<picture>`?

For **art direction** (a different crop or image at different breakpoints) and **format fallbacks** (AVIF, then WebP, then JPEG). The browser takes the first `<source>` whose `media` and `type` match; the inner `<img>` is required — it provides `alt`, sizing and the fallback.

```
<picture>
  <source type="image/avif" srcset="hero.avif">
  <source type="image/webp" srcset="hero.webp">
  <img src="hero.jpg" alt="Team at the summit" width="1200" height="800">
</picture>
```

→ *More:* §4.3 Responsive images — `srcset`, `sizes` and `<picture>`, §14.6 Q12

Q75. Which image format would you choose: JPEG, PNG, GIF, WebP, AVIF or SVG?

Photos: AVIF or WebP (much smaller than JPEG), with a JPEG fallback if needed. Screenshots or images needing lossless quality or transparency: PNG or lossless WebP. Logos, icons and illustrations: SVG (vector, so sharp at any size and tiny). Animations: a muted looping `<video>` (MP4/WebM) instead of a huge GIF.

→ *More:* §4.2 Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG

Q76. Raster vs vector images?

Raster images (JPEG, PNG, WebP, AVIF) are grids of pixels: they blur when scaled up and file size grows with dimensions. Vector images (SVG) are shapes described mathematically: crisp at any size, styleable with CSS, but unsuitable for photos.

→ *More:* §4.2 Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG, §4.8 SVG basics

Q77. ⋆ Why set `width` and `height` on images?

From the two attributes the browser computes the aspect ratio and reserves the right amount of space before the image downloads, so the text below doesn't jump when it arrives. That prevents layout shift (CLS). With `img { max-width: 100%; height: auto; }` the image still scales responsively.

Follow-up: What if the image is responsive with `width: 100%`? Keep the attributes anyway; with `height: auto` in CSS the browser uses their ratio for the reserved space.

→ *More:* §4.1 The `<img>` element, §14.6 Q46

Q78. How does `loading="lazy"` work, and when should you avoid it?

It defers loading an image or iframe until it gets near the viewport, saving bandwidth. Avoid it on the **LCP image** and anything above the fold — lazy-loading the hero image delays the page's main paint. For the hero use `fetchpriority="high"` instead. `decoding="async"` lets the browser decode the image off the critical path.

→ *More:* §4.4 Loading hints — `loading`, `decoding`, `fetchpriority`

Q79. Have you worked with retina (high-DPI) graphics? What techniques do you use?

High-DPI screens have a **device pixel ratio** (DPR) of 2 or 3: several physical pixels per CSS pixel, so a 1x image looks blurry. Techniques: `srcset` with `2x` or `w` descriptors, SVG for icons and logos, `image-set()` for CSS backgrounds, and icon fonts or CSS where possible. Don't send 3x images to everyone — let the browser choose.

→ *More:* §4.3 Responsive images — `srcset`, `sizes` and `<picture>`, §10.8 Responsive images recap (and `image-set()`)

Q80. ⋆ Canvas vs SVG?

SVG is **retained-mode**: shapes are DOM elements you can style, animate with CSS, attach events to, and that screen readers can read; it stays sharp at any size, but thousands of nodes get slow. `<canvas>` is **immediate-mode**: you draw pixels with JavaScript and the browser keeps no objects, so it handles many moving objects (games, big charts) quickly but is not accessible by default and must be redrawn for resizing or high-DPI.

Follow-up: Which would you pick for an interactive chart with 50 bars? SVG (accessible, styleable, crisp). For a live chart with 100,000 points, canvas (or WebGL).

→ *More:* §4.9 `<canvas>` — an introduction

Q81. Inline SVG vs `<img src="icon.svg">`?

Inline SVG becomes part of the DOM: you can style it with CSS (`fill: currentColor` to match text colour) and script it, but it isn't cached separately and bloats the HTML. `<img>` SVG is cached and simple but can't be styled from the page. For icon sets, an SVG sprite with `<use href="#icon">` is a middle ground.

→ *More:* §4.8 SVG basics

Q82. Are you familiar with styling SVG?

Yes: inline SVG elements accept CSS properties such as `fill`, `stroke`, `stroke-width`, `stroke-dasharray` (for line-drawing animations), `opacity` and transforms. `fill: currentColor` makes an icon inherit the text colour. Accessible SVG uses `role="img"` and a `<title>` (or `aria-label`); decorative SVG gets `aria-hidden="true"`.

→ *More:* §4.8 SVG basics

Q83. How do you include audio and video, and make them accessible?

`<video controls width height poster>` with one or more `<source type>` children, and `<track kind="captions" srclang="en" src="captions.vtt">` for captions (WebVTT files). Provide a transcript for audio. Browsers generally allow autoplay only when muted (`autoplay muted playsinline`), and moving content that plays for more than 5 seconds needs a pause control (WCAG 2.2.2).

→ *More:* §4.6 Audio and video — `<audio>`, `<video>`, `<source>`, `<track>`

Q84. What is an `<iframe>`, and how do you embed one safely?

It embeds another HTML page inside yours. Use `sandbox` (turns everything off; then allow only what's needed, e.g. `sandbox="allow-scripts"`), `allow` for permissions (camera, fullscreen), `referrerpolicy`, `loading="lazy"` and a `title` for screen readers. To stop *others* from framing your site (clickjacking), send `Content-Security-Policy: frame-ancestors 'self'`.

→ *More:* §4.7 Embedding other pages — `<iframe>`

Q85. How do two iframes, or a page and its iframe, communicate?

Same-origin frames can access each other's `window` directly. Cross-origin frames must use `postMessage`: `frame.contentWindow.postMessage(data, "https://child.example")`, and the receiver listens for `message` events and **checks** `event.origin` before trusting the data.

Browser storage and platform APIs

Q86. ⋆ Describe the difference between a cookie, `sessionStorage` and `localStorage`.

	Cookie	`localStorage`	`sessionStorage`
Sent to server	Yes, with every matching request	No	No
Size	~4 KB each	~5 MB per origin	~5 MB per origin
Lifetime	Until `Expires` / `Max-Age`, or session	Until cleared	Until the tab closes
Scope	Domain + path	Origin (scheme + host + port)	Origin **and** tab
JS access	`document.cookie` unless `HttpOnly`	Yes	Yes

Cookies are for server-readable state (sessions); Web Storage is synchronous, strings only, and for small client-side preferences.

Follow-up: Is `localStorage` shared between tabs? Yes, between all tabs of the same origin (and a `storage` event fires in the other tabs). `sessionStorage` is per tab. Both are blocked or partitioned in some private-browsing and third-party contexts.

→ *More:* §1.5 HTTP basics — requests and responses

Q87. Where should you store an authentication token?

Anything readable by JavaScript (`localStorage`, non-HttpOnly cookies) can be stolen by an XSS attack. The common recommendation is an `HttpOnly; Secure; SameSite=Lax` (or `Strict`) cookie — JavaScript can't read it, it's only sent over HTTPS, and `SameSite` blocks most CSRF (cross-site request forgery, where another site makes the browser send an authenticated request). Pair it with CSRF tokens for state-changing requests if needed.

Q88. What are the cookie attributes `HttpOnly`, `Secure`, `SameSite`, `Domain`, `Path`?

`HttpOnly`: hidden from JavaScript. `Secure`: HTTPS only. `SameSite=Strict|Lax|None`: whether the cookie is sent on cross-site requests (`Lax`, the default in Chromium, sends it on top-level link navigations; `None` requires `Secure`). `Domain` / `Path`: which URLs receive it. `Expires` / `Max-Age`: lifetime.

→ *More:* §1.5 HTTP basics — requests and responses

Q89. When would you use IndexedDB instead of `localStorage`?

`localStorage` is synchronous (it blocks the main thread), stores only strings and is small. IndexedDB is an asynchronous, transactional database in the browser that stores structured data and files (Blobs), with indexes and much larger quotas — use it for offline data, caches or large datasets. The Cache API (used with service workers) stores HTTP responses.

Q90. What are Web Components?

A set of standards for reusable, encapsulated custom elements: **Custom Elements** (`customElements.define('my-card', class extends HTMLElement {…})`), **Shadow DOM** (a private DOM subtree whose styles don't leak in or out), and `<template>` / `<slot>` (inert markup to clone, and placeholders for the user's content). They work in any framework or none.

Q91. What is the Shadow DOM, and how do you style it?

A hidden DOM tree attached to an element (`attachShadow({mode: 'open'})`). Page CSS selectors don't reach inside, and its styles don't leak out — but **inherited properties** (colour, font) and **custom properties** do pass through. Style it from outside via CSS variables, `::part()` (parts the component exposes) and `::slotted()` / `:host` from inside.

Q92. What are Web Workers and service workers?

A **Web Worker** runs JavaScript on a background thread (no DOM access), talking to the page via `postMessage` — for heavy computation that would freeze the UI. A **service worker** is a special worker that sits between the page and the network: it can intercept requests, serve cached responses (offline support), and receive push messages. It requires HTTPS.

Q93. What are server-sent events and WebSockets?

Server-sent events (`EventSource`): a one-way stream of text messages from server to browser over plain HTTP, with automatic reconnection — good for notifications and live feeds. **WebSockets**: a full two-way connection for low-latency messages in both directions — chat, games, collaborative editing.

Selectors, specificity, the cascade and inheritance

Q94. What are the three ways to add CSS to a page, and which is preferred?

An external stylesheet (`<link rel="stylesheet">`, cached and shared across pages — the default), an internal `<style>` block (good for critical CSS or one-off pages), and the inline `style` attribute (highest priority among normal declarations, hard to override and to maintain, no media queries or pseudo-classes). `@import` inside CSS also works but delays loading because the browser discovers the file late.

→ *More:* §6.1 Syntax and adding CSS to a page

Q95. ⋆ What is CSS selector specificity, and how is it calculated?

Specificity is a score that decides between conflicting declarations from the same origin and layer. Count three columns **(A, B, C)**: A = ID selectors, B = classes, attribute selectors and pseudo-classes, C = type (element) selectors and pseudo-elements. Compare A first, then B, then C — so `#nav` (1,0,0) beats `.a.b.c.d.e.f.g.h.i.j.k` (0,11,0). The universal selector `*` and combinators add nothing. Inline styles beat any selector; `!important` works at a different level (origin), not as extra specificity.

Follow-up: What specificity does `style="…"` have? It's not part of the A-B-C score; inline styles are a separate cascade step above all selectors. Only `!important` declarations (or animations, for animated values) beat it.

→ *More:* §6.4 Specificity, §14.6 Q14

Q96. What is the specificity of `:is()`, `:not()`, `:has()` and `:where()`?

`:is()`, `:not()` and `:has()` take the specificity of their **most specific argument** — `:is(#a, .b)` counts as an ID even when `.b` is what matched. `:where()` always has **zero** specificity, which makes it ideal for defaults and resets that should be easy to override.

→ *More:* §6.4 Specificity, §13.2 `:is()` and `:where()`, §14.6 Q20

Q97. ⋆ What decides which declaration wins? Explain the cascade.

The cascade sorts competing declarations by, in order: **origin and importance** (user-agent, user, author styles; `!important` reverses the order), **context** (shadow DOM), **inline style**, **cascade layer**, **specificity**, **scope proximity** (for `@scope`), and finally **order of appearance** (the last one wins). Only if one step ties does the next one count.

Follow-up: Where do animations and transitions fit in the cascade? Declarations from running CSS animations override all normal declarations; running transitions override even `!important` ones.

→ *More:* §6.5 The cascade and origins, §14.6 Q15

Q98. When should you use `!important`?

Rarely: for utility classes that must always win (`.hidden { display: none !important }`), for overriding third-party inline styles you can't change, and in user stylesheets for accessibility. Using it to win specificity fights creates an escalating war that only more `!important` can win. Fix the cause instead: lower specificity, use cascade layers. Note that `!important` in an earlier (lower) layer beats `!important` in a later layer.

→ *More:* §6.5 The cascade and origins, §13.4 Cascade layers (`@layer`)

Q99. ⋆ Pseudo-class vs pseudo-element?

A **pseudo-class** (one colon) selects an element in a particular state or position: `:hover`, `:focus-visible`, `:checked`, `:nth-child(2)`, `:has()`. A **pseudo-element** (two colons) selects or creates a *part* of an element that isn't in the DOM: `::before`, `::after`, `::first-line`, `::placeholder`, `::marker`, `::selection`, `::backdrop`.

Follow-up: Why the double colon? CSS3 introduced `::` to distinguish pseudo-elements; browsers still accept the old single colon for `:before`, `:after`, `:first-line` and `:first-letter` for compatibility.

→ *More:* §6.3 Pseudo-classes and pseudo-elements, §14.6 Q16

Q100. What are `::before` and `::after` used for? Why doesn't mine show up?

They insert a generated box as the first/last child of the element's content — decorative icons, quotes, tooltips' arrows, clearfixes, overlays. They only render if `content` is set (`content: ""` at minimum). Replaced elements such as `<img>`, `<video>` and `<iframe>` don't render them, and most form controls don't either — the exception is a checkbox or radio with `appearance: none`, which current browsers do decorate (the toggle switch in Q230 relies on it). Text in `content` is read by most screen readers, so don't put essential content in it.

→ *More:* §6.3 Pseudo-classes and pseudo-elements

Q101. ⋆ `:nth-child()` vs `:nth-of-type()`?

`:nth-child(2)` matches an element that is the 2nd child of its parent, *of any type*, and also matches the rest of the selector. `:nth-of-type(2)` counts only siblings of the same element type. For `<div><h2/><p/><p/></div>`, `p:nth-child(2)` is the first `<p>`; `p:nth-of-type(2)` is the second `<p>`. Modern `:nth-child(2 of .item)` counts only elements matching `.item`.

Follow-up: Write a selector for every third item starting with the second: `li:nth-child(3n + 2)`. For the last two items: `li:nth-last-child(-n + 2)`.

→ *More:* §6.3 Pseudo-classes and pseudo-elements

Q102. What do the combinators ` `, `>`, `+` and `~` mean?

Descendant (`nav a` — any depth), child (`ul > li` — direct children only), next sibling (`h2 + p` — the paragraph immediately after an h2), subsequent sibling (`h2 ~ p` — every later sibling paragraph).

→ *More:* §6.2 Selectors

Q103. Name the attribute selectors.

`[href]` (has it), `[type="email"]` (exactly), `[class~="card"]` (word in a space-separated list), `[lang|="en"]` (`en` or `en-…`), `[href^="https"]` (starts with), `[href$=".pdf"]` (ends with), `[href*="example"]` (contains); add `i` before `]` for case-insensitive matching.

→ *More:* §6.2 Selectors

Q104. ⋆ Explain how a browser determines which elements match a selector. Why does it read right to left?

Browsers match selectors **from right to left**: for `.nav ul li a`, they start with each `<a>` (the **key selector**) and walk up the ancestors to check the rest. Most elements fail the rightmost part immediately, so this is fast. That's why a very generic key selector with a long chain (`.sidebar *`) does more work. In practice modern engines are fast enough that selector performance rarely matters compared with layout costs.

Follow-up: So is `.nav a` faster than `a`? Practically irrelevant. Write selectors for readability and low specificity; worry about layout and paint costs instead.

→ *More:* §6.2 Selectors

Q105. ⋆ Which CSS properties inherit, and how do you control inheritance?

Mostly text-related properties inherit: `color`, `font-*`, `line-height`, `letter-spacing`, `text-align`, `visibility`, `cursor`, and custom properties. Box properties don't: `margin`, `padding`, `border`, `width`, `background`, `display`. The keywords `inherit` (take the parent's value), `initial` (the property's spec default), `unset` (inherit if inherited, else initial) `revert` (roll back to the previous origin — usually the browser's default stylesheet) and `revert-layer` (roll back to the previous cascade layer) work on every property; `all: unset` resets everything.

→ *More:* §6.6 Inheritance, §14.6 Q17

Q106. ⋆ What is the difference between "resetting" and "normalizing" CSS? Which would you choose?

A **reset** removes browser defaults (margins, list styles, heading sizes) so you start from zero; a **normalize** keeps useful defaults but makes them consistent across browsers and fixes bugs. Today most teams use a small **modern reset**: `box-sizing: border-box` everywhere, zero body margin, responsive media (`img { max-width: 100%; display: block }`), `font: inherit` on form controls — keeping semantics-friendly defaults like list bullets where wanted.

Follow-up: What's in your usual reset? `box-sizing: border-box`, `margin: 0` on body/headings/paragraphs, `img, svg, video { display: block; max-width: 100% }`, `input, button, textarea, select { font: inherit }`, `line-height: 1.5` on body, and `overflow-wrap: break-word` on text.

→ *More:* §13.13 Resets and normalize

Q107. How would you approach fixing browser-specific styling issues?

Reproduce it and identify the engine and version; check MDN / caniuse / Baseline for support; use `@supports` for feature detection and a fallback; use Autoprefixer (a PostCSS tool) instead of hand-written vendor prefixes; use a reset/normalize; and as a last resort, isolate a targeted fix with a comment and a removal date. Avoid user-agent sniffing (guessing the browser from its name string).

→ *More:* §13.12 Preprocessors (Sass) and PostCSS

Q108. How do you serve your pages for feature-constrained (older) browsers?

Progressive enhancement: build a working baseline in plain HTML and CSS, then layer on features where they're supported. Tools: `@supports (display: grid) { … }`, properties browsers ignore if unknown (write the fallback first, the modern value second), Autoprefixer and Browserslist, polyfills only where needed, and testing on real old devices. **Graceful degradation** is the reverse: build for modern browsers and make sure it still works acceptably when features are missing.

→ *More:* §10.6 Container queries, §13.12 Preprocessors (Sass) and PostCSS

Q109. Can you give an example of an `@media` rule other than `screen`?

`@media print { nav, .ads { display: none } a::after { content: " (" attr(href) ")" } }` for printing. Others: `@media (prefers-color-scheme: dark)`, `(prefers-reduced-motion: reduce)`, `(hover: hover) and (pointer: fine)` for mouse users, `(orientation: landscape)`, `(forced-colors: active)` for Windows high-contrast mode.

→ *More:* §10.3 Media queries, §10.5 User-preference media queries

---

Box model, `display` and units

Q110. ⋆ Explain the CSS box model.

Every element generates a rectangular box made of four layers, from inside out: **content** (text, images), **padding** (space inside the border, takes the background), **border**, and **margin** (transparent space outside, between boxes). By default `width` and `height` set only the content box, so a `width: 200px` box with `20px` padding and a `5px` border is 250 px wide.

Follow-up: What is the margin of a box doing to its background? Nothing — backgrounds paint under content and padding (and border by default, see `background-clip`), never under the margin.

→ *More:* §7.1 The box model, §14.6 Q23

Q111. ⋆ What does `* { box-sizing: border-box; }` do, and what are its advantages?

It makes `width` and `height` include padding and border, so `width: 200px` stays 200 px no matter what padding you add. Layout maths becomes predictable (`width: 50%` plus padding no longer overflows), which is why almost every reset includes it — usually as `*, *::before, *::after { box-sizing: border-box; }`. The other value, `content-box`, is the default.

Follow-up: Does `border-box` include the margin? No. Margins are always outside the box's size.

→ *More:* §7.2 `box-sizing`, §14.6 Q23

Q112. ⋆ What is the CSS `display` property? Give some examples.

It sets the box's **outer** role (how it takes part in its parent's layout: `block` or `inline`) and its **inner** layout for its children (`flow`, `flex`, `grid`, `table`). Common values: `block`, `inline`, `inline-block`, `flex`, `inline-flex`, `grid`, `inline-grid`, `flow-root`, `contents` (the box disappears, children stay), `none`, `table`, `list-item`.

→ *More:* §7.4 `display`

Q113. ⋆ Block vs inline vs inline-block?

Block (`div`, `p`, `h1`): starts on a new line, fills the available width, respects `width`/`height` and all margins. **Inline** (`span`, `a`, `strong`): flows within text, ignores `width`/`height`, and vertical

margins/padding don't push other lines away. **Inline-block**: flows in the line like text but respects `width`, `height`, padding and margins like a block — useful for buttons and badges.

Follow-up: Can you set `padding` on an inline element? Yes, and it's painted, but vertical padding doesn't push neighbouring lines apart, so it can overlap the lines above and below.

→ *More:* §7.4 `display`, §14.6 Q3

Q114. Can you change an inline element into a block-level element?

Yes — `display` is CSS, independent of the HTML element. `a { display: block }` makes the whole link area clickable. But the HTML content model (which elements may contain which) doesn't change: `<span>` still can't legally contain a `<div>` in the markup. (Since HTML5, `<a>` may wrap block content.)

→ *More:* §7.4 `display`

Q115. ⋆ `display: none` vs `visibility: hidden` vs `opacity: 0`?

`display: none` removes the box: no space, not rendered, hidden from screen readers, not focusable. `visibility: hidden` keeps the space but hides it (also from screen readers and focus); a child can override it with `visibility: visible`. `opacity: 0` keeps space and stays interactive — clickable, focusable and announced. `opacity` animates smoothly and `visibility` flips discretely at the end of a transition; `display: none` can now be part of a transition too with `transition-behavior: allow-discrete` plus `@starting-style` (see Q182).

Follow-up: Which of the three still receives clicks? Only `opacity: 0`. Add `pointer-events: none` if it shouldn't, and remember it's still announced and focusable.

→ *More:* §3.4 The accessibility tree, §12.1 Transitions

Q116. What is a replaced element?

An element whose content comes from outside CSS — `<img>`, `<video>`, `<iframe>`, `<input>`, `<canvas>` — so it has an **intrinsic size** (and often an aspect ratio). CSS sizes the box but doesn't lay out the inside; `object-fit` controls how the content fills it, and `::before`/`::after` don't apply.

→ *More:* §7.4 `display`, §11.9 `object-fit` and `object-position`

Q117. Why is there a small gap under my image, or between inline-block elements?

Under an image: images are inline by default and sit on the text **baseline**, leaving room for letter descenders (g, y). Fix with `img { display: block }` or `vertical-align: middle`. Between inline-blocks: the white space (newlines) in the HTML is rendered as a space character. Fix by using flexbox or grid with `gap` instead.

→ *More:* §7.4 `display`

Q118. ⋆ Explain `px`, `em`, `rem`, `%`, `vw`/`vh` — when do you use each?

`px`: an absolute CSS pixel (1/96 inch at arm's length; not a device pixel) — borders, shadows. `em`: relative to the element's own `font-size` (for the `font-size` property itself, the parent's) — padding that should scale with the text. `rem`: relative to the root (`<html>`) font size — font sizes and spacing that respect the user's browser setting. `%`: relative to the parent (width-based for padding and margin, even vertical ones). `vw`/`vh`: 1% of the viewport's width/height — full-screen sections, fluid type.

Follow-up: What does `1px` mean on a phone with DPR 3? One CSS pixel is drawn with 3×3 device pixels, so CSS sizes look the same physical size on all screens.

→ *More:* §6.7 Units, §14.6 Q18

Q119. `em` vs `rem` — what is the compounding problem?

Nested `em` font sizes multiply: a list item with `font-size: 1.2em` inside another one becomes 1.44 × the base, and so on. `rem` always refers to the root, so there is no compounding. Use `rem` for font sizes, `em` for things that should scale with the local text size (icon size, button padding, media query breakpoints).

→ *More:* §6.7 Units

Q120. Why should font sizes use `rem` rather than `px`?

Users can set a larger default font size in the browser; `rem` scales with it, `px` ignores it (page zoom scales both, but the font-size setting doesn't). Fixed `px` text can fail WCAG 1.4.4 (resize text)

for users who rely on that setting.

→ *More:* §6.7 Units

Q121. `100vh` on mobile is too tall — why, and what are `svh`, `lvh` and `dvh`?

On mobile the browser's toolbar shows and hides as you scroll; `vh` uses the **largest** viewport (toolbar hidden), so a `100vh` section is cut off behind the toolbar. `svh` = small viewport (toolbars shown), `lvh` = large, `dvh` = dynamic (changes as the toolbar moves, which can cause jumps). Use `min-height: 100svh` (or `100dvh`) for full-screen sections.

→ *More:* §10.2 The viewport meta tag and viewport units

Q122. Why should `line-height` be unitless?

A unitless `line-height: 1.5` is inherited as the *ratio*, so each child recomputes it from its own font size. `line-height: 1.5em` or `24px` is computed once on the parent and inherited as a fixed length, so a larger heading inside gets cramped, overlapping lines.

→ *More:* §11.3 Text properties

Q123. What is the difference between `border`, `outline` and `box-shadow`?

A border is part of the box model — it takes space and changes the size. An outline and a box-shadow are drawn outside the box without affecting layout. Outlines follow `border-radius` in modern browsers and are visible in Windows high-contrast (forced-colours) mode, which is why they are the right tool for focus rings; box-shadows disappear in that mode.

→ *More:* §7.1 The box model, §11.6 Borders, radius and outlines

Q124. What are `min()`, `max()` and `clamp()`?

Math functions that pick a value at render time: `width: min(100%, 60rem)` (never wider than 60rem, never overflow), `max(1rem, 2vw)`, and `clamp(MIN, PREFERRED, MAX)` — the preferred value bounded on both sides, e.g. `font-size: clamp(1rem, 0.8rem + 1vw, 1.5rem)`. They cut out many media queries.

→ *More:* §10.7 Fluid typography with `clamp()`, §14.6 Q37

Positioning, `z-index`, stacking contexts, floats and BFC

Q125. ⋆ Explain `static`, `relative`, `absolute`, `fixed` and `sticky` positioning.

`static` (the default): normal flow, offsets (`top`, `left`…) ignored. `relative`: stays in the flow and keeps its space, but is shifted visually by offsets; also becomes the containing block for absolute children. `absolute`: removed from the flow, placed relative to the nearest **positioned** ancestor (any value except `static`) or the initial containing block. `fixed`: removed from the flow, placed relative to the viewport — stays during scroll. `sticky`: behaves like `relative` until the scroll position crosses a threshold (`top: 0`), then sticks within its parent's box.

Follow-up: Does `position: relative` with no offsets do anything? Visually nothing, but it creates a containing block for absolute children and lets `z-index` apply.

→ *More:* §7.6 Positioning, §14.6 Q25

Q126. What is the containing block for an absolutely positioned element?

The padding box of the nearest ancestor whose `position` isn't `static` — or that has a `transform`, `filter`, `perspective`, `contain: layout/paint`, or `will-change` for those. If there is none, the **initial containing block** (the viewport-sized box at the document's start). That's why the usual pattern is `position: relative` on the parent, `position: absolute` on the child.

→ *More:* §7.6 Positioning

Q127. ⋆ Why doesn't my `position: sticky` work?

Common causes: no threshold set (`top: 0` is required); an ancestor has `overflow: hidden`, `auto` or `scroll`, which makes *it* the scrolling container to stick within; the parent is no taller than the sticky element, so there's no room to stick; or in a flex/grid container the item is stretched to full height (add `align-self: start`).

→ *More:* §7.6 Positioning

Q128. Why is my `position: fixed` element scrolling with the page (or positioned wrongly)?

An ancestor has a `transform`, `filter`, `perspective`, `backdrop-filter`, `contain` or `will-change: transform`. Those make the ancestor the containing block for fixed descendants, so "fixed" becomes relative to that box. Move the element outside that ancestor, remove the property, or use the top layer (`<dialog>`, `popover`).

→ *More:* §7.6 Positioning

Q129. ⋆ Is there any reason you'd use `translate()` instead of absolute positioning, or vice versa?

`translate()` moves an element visually **without** changing layout: its original space is kept, nothing reflows, and the change can be done on the GPU compositor — smooth for animation. Changing `top` / `left` triggers layout each frame. Use absolute positioning to *place* something in layout (a badge in a corner, taken out of the flow); use `transform: translate()` to *move or animate* it. Note that a transform creates a stacking context and a containing block for fixed children.

→ *More:* §12.3 2D transforms, §12.7 Performance: animate `transform` and `opacity`

Q130. ⋆ Describe `z-index` and how a stacking context is formed.

`z-index` sets the stacking order of positioned elements (and flex/grid items) along the z-axis. It only compares elements **within the same stacking context** — a group painted as one layer. A new stacking context is created by: the root element; positioned elements with a `z-index` other than `auto`; `position: fixed` or `sticky`; flex/grid items with a `z-index`; `opacity` below 1; `transform`, `filter`, `backdrop-filter`, `perspective`, `clip-path`, `mask`; `mix-blend-mode` other than `normal`; `isolation: isolate`; `contain: paint` / `layout`; `will-change` for any of those; and elements in the top layer.

Follow-up: Does `z-index` work on a flex item without `position`? Yes — flex and grid items accept `z-index` even when they're `position: static`.

→ *More:* §7.8 `z-index` and stacking contexts

Q131. ⋆ I set `z-index: 9999` on a modal and it still appears behind the header. Why?

The modal is inside a parent that forms its own stacking context (for example, a parent with `transform`, `opacity < 1` or `position: relative; z-index: 1`). Everything inside that parent is stacked as one unit, so 9999 only beats its siblings inside the parent; the parent as a whole sits below the header. Fix: raise the *ancestor's* z-index, remove the property that created the context, render the modal at the end of `<body>` (a "portal"), or use `<dialog>` / `popover`, which go into the **top layer** above every stacking context.

Follow-up: How do you debug it quickly? In DevTools, walk up the modal's ancestors and look for `transform`, `opacity`, `filter`, `will-change`, `isolation`, or `position` + `z-index`; the Layers panel and the 3D view also show stacking contexts.

→ *More:* §7.8 `z-index` and stacking contexts, §14.6 Q26

Q132. Without any `z-index`, in what order are elements painted?

Within one stacking context, back to front: the context's own background and borders; children with negative `z-index`; in-flow block boxes; floats; inline content; positioned descendants with `z-index: auto` or `0` in DOM order; then positive `z-index` children. So a later positioned element covers an earlier one, and any positioned element covers non-positioned ones.

→ *More:* §7.8 `z-index` and stacking contexts

Q133. ⋆ Describe floats and how they work.

`float: left|right` takes a box out of normal flow and pushes it to the side of its container; following **inline content (text) wraps around it**, but block boxes behave as if it weren't there (their backgrounds slide underneath). The parent doesn't grow to contain floated children ("collapses"). Floats were used for whole page layouts before flexbox and grid; today they are for what they were designed for — wrapping text around an image (optionally with `shape-outside`).

Follow-up: What does `shape-outside` add? It lets text wrap around a non-rectangular shape — `shape-outside: circle(50%)` on a floated round image.

→ *More:* §7.9 Floats (legacy)

Q134. ⋆ What are the various clearing techniques, and which is appropriate when?

`clear: both` on an element after the floats moves it below them. To make a parent contain its floats: `display: flow-root` on the parent (the modern, side-effect-free choice — it creates a BFC); `overflow: auto` / `hidden` (also a BFC but may clip shadows or show scrollbars); or the legacy **clearfix** — a `::after` pseudo-element with `content: ""; display: table; clear: both;`. In new code use `flow-root` — or don't use floats for layout at all.

→ *More:* §7.9 Floats (legacy)

Q135. ⋆ What is a block formatting context (BFC), and how is one created?

A BFC is an independent layout region in which block boxes are laid out vertically, isolated from the outside. Inside it: floats are contained (the parent grows around them), margins of children don't collapse through it, and the BFC box doesn't overlap floats outside it. Created by: the root `<html>`, `display: flow-root`, floats, absolute/fixed positioning, `inline-block`, table cells, flex and grid **items**, `overflow` other than `visible` / `clip`, `contain: layout` / `paint`, and multi-column containers.

Follow-up: Give a concrete reason you created a BFC recently: to stop a child's top margin leaking out of its parent card, or to stop text from wrapping under a floated image (making the text column a BFC).

→ *More:* §7.5 Normal flow and formatting contexts, §14.6 Q27

Q136. ⋆ What is margin collapsing? When does it happen and how do you stop it?

Vertical margins of block boxes in normal flow can combine into one margin equal to the larger of them. With a negative margin involved, the largest positive and the most negative are added (30px and −10px → 20px); if both are negative, the most negative wins. It happens between **adjacent siblings** (20px + 30px → 30px), between a **parent and its first/last child** when nothing separates them (the child's top margin "leaks out" of the parent), and through **empty blocks**. It never happens horizontally, nor in flex or grid containers. Stop it with padding or a border on the parent, `display: flow-root`, or by using flex/grid with `gap`.

Follow-up: Why does a heading's top margin push the whole card down instead of creating space inside it? The heading's margin collapsed through the card's top edge. Give the card padding or `display: flow-root`.

→ *More:* §7.3 Margin collapsing, §14.6 Q24

Q137. What is `overflow`, and what is the difference between `hidden`, `clip`, `auto` and `scroll`?

It controls content that doesn't fit its box. `visible` (default) lets it spill out. `hidden` clips it but the box is still a scroll container (scrollable by script, creates a BFC, breaks `position: sticky` inside). `clip` clips without creating a scroll container (sticky still works). `auto` shows scrollbars only when needed; `scroll` always reserves them.

→ *More:* §7.7 Overflow

Q138. How do you truncate text with an ellipsis — single line and multi-line?

```
.one-line { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.three-lines { display: -webkit-box; -webkit-box-orient: vertical;
               -webkit-line-clamp: 3; line-clamp: 3; overflow: hidden; }
```

The single-line version needs a constrained width (and `min-width: 0` on a flex item). The multi-line version uses the prefixed `-webkit-line-clamp`, which works in all major browsers.

→ *More:* §11.3 Text properties

Flexbox

Q139. ⋆ Explain flexbox and its main properties.

Flexbox is a **one-dimensional** layout mode: `display: flex` on a container lays its direct children (flex items) out along a **main axis** (a row by default) and aligns them on the **cross axis**. Container properties: `flex-direction`, `flex-wrap`, `justify-content` (main axis), `align-items` and `align-content` (cross axis), `gap`. Item properties: `flex-grow`, `flex-shrink`, `flex-basis` (shorthand `flex`), `align-self`, `order`.

→ *More:* §8.1 Flex container and flex items, §8.2 The two axes, `flex-direction` and `order`

Q140. ⋆ What does `flex: 1` mean? What about `flex: auto`, `flex: none` and `flex: 0 0 200px`?

`flex` is shorthand for grow, shrink, basis. `flex: 1` = `1 1 0%`: start from zero and share *all* space equally — equal-width columns. `flex: auto` = `1 1 auto`: start from content size and share the *leftover* space, so items with more content end up wider. `flex: none` = `0 0 auto`: rigid, content-sized. `flex: 0 0 200px`: exactly 200 px, never grows or shrinks.

Follow-up: Why might `flex: 1` columns still be unequal? Because of the automatic minimum size: an item with a long unbreakable word can't shrink below it. Add `min-width: 0`.

→ *More:* §8.4 The `flex` shorthand and minimum sizes, §14.6 Q28

Q141. Explain `flex-grow`, `flex-shrink` and `flex-basis`.

`flex-basis` is the item's starting size on the main axis (`auto` = use `width`/content). If there is free space left, it's shared in proportion to `flex-grow`. If items overflow, the overflow is removed in proportion to `flex-shrink` **multiplied by each item's basis** (so bigger items shrink more). Items never shrink below their minimum size (see the next question).

→ *More:* §8.3 `flex-grow`, `flex-shrink` and `flex-basis`

Q142. ⋆ Why does a flex item with long text (or a long URL, or a `<pre>`) overflow instead of shrinking?

Flex items have `min-width: auto` by default, which means "don't shrink below my content's minimum width" — the longest word or the unbreakable string. Set `min-width: 0` (or `min-height: 0` in a column) on the item, and add `overflow-wrap: anywhere` or `text-overflow: ellipsis` if the text itself must break. Grid items have the same rule, which is why `1fr` columns can blow out too.

Follow-up: Does `overflow: hidden` on the item also fix it? Yes — a scroll container's automatic minimum size is 0. `overflow: clip` does not, because it doesn't create a scroll container.

→ *More:* §8.4 The `flex` shorthand and minimum sizes, §14.6 Q29

Q143. `justify-content` vs `align-items` vs `align-content`?

`justify-content` distributes items along the **main** axis (`flex-start`, `center`, `space-between`, `space-around`, `space-evenly`). `align-items` aligns items on the **cross** axis within their line (`stretch` — the default, `center`, `flex-start`, `baseline`). `align-content` distributes the **lines** themselves when wrapping produces several lines; it has no effect on a single-line container.

→ *More:* §8.5 Alignment

Q144. Why doesn't `justify-content: center` work in my `flex-direction: column` container?

Changing the direction swaps the axes: in a column, `justify-content` acts vertically and `align-items` horizontally. Also, vertical centering needs the container to have a height taller than its content.

→ *More:* §8.2 The two axes, `flex-direction` and `order`, §8.5 Alignment

Q145. How do you push one flex item to the far end (for example, a "Log out" link to the right)?

Give it `margin-left: auto` (or `margin-inline-start: auto`). In a flex container, leftover space is handed to `auto` margins before `justify-content` runs, so that margin grows as large as possible and shoves the item — and everything after it — to the end. No wrapper needed.

→ *More:* §8.5 Alignment

Q146. What does `gap` do, and why is it better than margins between items?

`gap` (`row-gap`, `column-gap`) adds space only *between* flex or grid items — never at the outer edges — so you don't need `:last-child { margin: 0 }` hacks, and it works correctly when items wrap. Supported in flexbox in all browsers since 2021.

→ *More:* §8.6 Wrapping and `gap`

Q147. Why is using `order` (or `flex-direction: row-reverse`) an accessibility concern?

They change only the *visual* order. Keyboard Tab order and screen-reader reading order still follow the DOM, so focus jumps around the screen unpredictably — WCAG 1.3.2 (Meaningful Sequence) and 2.4.3 (Focus Order). Change the DOM order instead when the order matters.

→ *More:* §8.2 The two axes, `flex-direction` and `order`

Q148. Why does the last row of my wrapping flex layout look wrong?

With `flex-wrap: wrap` and `flex-grow`, each line is sized independently, so a lonely last item stretches to full width (or `space-between` spreads two items to the far edges). Flexbox has no columns to align to. If items must line up in columns, use grid (`repeat(auto-fill, minmax(…))`).

→ *More:* §8.6 Wrapping and `gap`

CSS Grid

Q149. ⋆ Explain CSS Grid's main concepts.

Grid is a **two-dimensional** layout: `display: grid` defines **tracks** (rows and columns, with `grid-template-columns` / `grid-template-rows`), separated by numbered **lines**; items are placed into **cells** or **areas** spanning several cells. `gap` adds gutters. Items can be placed explicitly (`grid-column: 1 / 3`, `grid-area: header`) or flow automatically (auto-placement).

→ *More:* §9.1 Grid vocabulary, §9.2 Defining tracks

Q150. What is the `fr` unit? How is it different from `%`?

`fr` is a fraction of the **free space left after** fixed tracks and gaps are subtracted: `200px 1fr 2fr` gives the second column a third and the third column two thirds of what remains. `%` is relative to the whole container and ignores gaps, so `50% 50%` plus a `gap` overflows, while `1fr 1fr` doesn't.

Follow-up: What does `fr` do when the content is bigger than the share? The track grows to its minimum content size (see the `1fr` overflow question), so tracks become unequal unless you use `minmax(0, 1fr)`.

→ *More:* §9.2 Defining tracks

Q151. Why does my `1fr` column overflow when it contains a long word or a wide image?

`1fr` really means `minmax(auto, 1fr)` — the minimum is the content's minimum size, so the column refuses to shrink below the longest unbreakable item. Use `minmax(0, 1fr)` (or `min-width: 0` on the item) to allow it to shrink.

→ *More:* §9.2 Defining tracks

Q152. ⋆ `auto-fill` vs `auto-fit`?

Both create as many columns as fit in `repeat(auto-fill|auto-fit, minmax(200px, 1fr))`. With **`auto-fill`**, leftover empty tracks are kept, so a few items stay at their minimum width with empty space after them. With **`auto-fit`**, empty tracks collapse to zero and the `1fr` lets the existing items stretch to fill the row. They only look different when there are fewer items than columns.

Follow-up: Which is better for a product grid that is sometimes nearly empty (one or two results)? `auto-fill`: items keep a consistent card width. `auto-fit` would stretch a single result across the whole row. (A lone item in the last row of a *full* grid looks the same with both, because the tracks already exist.)

→ *More:* §9.3 `repeat()`, `auto-fill` and `auto-fit`, §14.6 Q32

Q153. How do you build a responsive card grid with no media queries?

```
.cards {
  display: grid; gap: 1rem;
  grid-template-columns: repeat(auto-fill, minmax(min(16rem, 100%), 1fr));
}
```

The browser fits as many 16rem columns as it can and shares the leftover space; `min(16rem, 100%)` stops a single column from overflowing on very narrow screens. This "RAM" pattern (Repeat, Auto, Minmax) is one of the most common interview tasks.

→ *More:* §9.9 Patterns, §9.3 `repeat()`, `auto-fill` and `auto-fit`

Q154. What is `grid-template-areas`?

A way to name regions with an ASCII-art map and place items by name:

```
.page { display: grid; grid-template-columns: 200px 1fr;
  grid-template-areas: "head head" "nav main" "foot foot"; }
header { grid-area: head; } nav { grid-area: nav; } main { grid-area: main; } footer { grid-area: foot; }
```

Changing the layout at a breakpoint is just redefining the map. Each area must be a rectangle; `.` marks an empty cell.

→ *More:* §9.4 Placing items

Q155. What are the explicit and implicit grid?

The **explicit grid** is the tracks you defined with `grid-template-*`. When items are placed outside it, or there are more items than cells, the browser adds **implicit** tracks, sized by `grid-auto-rows` / `grid-auto-columns` (default `auto`). `grid-auto-flow: dense` lets auto-placement backfill holes (but changes visual order).

→ *More:* §9.5 The implicit grid and auto-placement

Q156. What is subgrid, and what problem does it solve?

`grid-template-columns: subgrid` (or rows) on a grid item makes its own children use the **parent's** tracks instead of new ones. Classic use: a row of cards where each card's title, body and button line up across cards even though their content lengths differ. Baseline since September 2023.

→ *More:* §9.7 Subgrid, §14.6 Q33

Q157. ⋆ What is the difference between Flexbox and Grid? When would you use one over the other?

Flexbox is **one-dimensional** and **content-out**: items size themselves from their content, and you control one row (or column) at a time — navbars, toolbars, a row of tags, centring one thing. Grid is **two-dimensional** and **layout-in**: you define the rows and columns, then place content into them — page layouts, card galleries, forms with aligned labels. They combine: a grid page whose header uses flex.

Follow-up: Can grid do what flexbox does? Mostly, but flexbox is simpler when items should size to their content and wrap naturally (tags, buttons); grid is better when items must line up in both directions.

→ *More:* §9.8 Grid vs Flexbox, §14.6 Q31

Q158. Have you used a grid system (Bootstrap's 12 columns, etc.)? What do you prefer now?

Framework grid systems used floats, then flexbox, with classes like `col-md-6`. Native CSS Grid makes most of them unnecessary: `grid-template-columns: repeat(12, 1fr)` gives a 12-column grid in one line, and `auto-fit/minmax` patterns remove many breakpoints. Design systems still often keep a column grid for consistent spacing.

→ *More:* §9.9 Patterns

Responsive design: media queries, container queries and fluid sizing

Q159. What is responsive design?

One codebase whose layout adapts to any screen size and input method, using fluid layouts (flex, grid, percentages), flexible media (`max-width: 100%`, `srcset`), and media or container queries. The core ideas came from Ethan Marcotte (2010): fluid grids, flexible images and media queries.

→ *More:* §10.1 What responsive design is

Q160. ⋆ What is the difference between responsive and mobile-first — and between fixed, fluid, adaptive and responsive layouts?

Mobile-first is a *strategy for writing* responsive CSS: base styles target small screens, and `min-width` media queries add layout for larger ones (simpler CSS, phones download less override CSS). **Fixed** layouts have a set pixel width; **fluid** use percentages and stretch; **adaptive** serve a few fixed designs chosen at breakpoints; **responsive** are fluid *and* use breakpoints to rearrange.

→ *More:* §10.1 What responsive design is, §10.4 Mobile-first in practice, §14.6 Q35

Q161. How do media queries work? How do you choose breakpoints?

`@media (min-width: 48em) { … }` applies rules when the condition is true (here, viewport at least 48em wide). Media Queries Level 4 adds range syntax: `@media (48em <= width < 64em)`. Choose breakpoints where **your content** breaks (a line gets too long, a nav no longer fits), not at specific device sizes. `em` breakpoints respond to the user's font-size setting.

→ *More:* §10.3 Media queries

Q162. ⋆ What are container queries, and how do they differ from media queries?

Media queries react to the **viewport**. Container queries react to the size of an **ancestor container**, so a component adapts to the space it's given — the same card is horizontal in a wide main column and stacked in a narrow sidebar.

```
.card-wrap { container-type: inline-size; }
@container (min-width: 30rem) { .card { display: grid; grid-template-columns: 10rem 1fr; } }
```

Size queries are Baseline since 2023; container units like `cqi` (1% of the container's inline size) go with them.

Follow-up: Why does a container need `container-type`? Querying a size requires the container's size not to depend on its children (size containment), otherwise there'd be a loop. `inline-size` contains only the width, so height can still grow with content.

→ *More:* §10.6 Container queries, §14.6 Q36

Q163. What are style queries?

`@container style(--variant: compact) { … }` applies styles when the container's custom property has a given value — a way to theme or switch component variants from a parent. Style queries on custom properties became Baseline in 2026.

→ *More:* §10.6 Container queries

Q164. How does `clamp()` create fluid typography?

`font-size: clamp(1rem, 0.75rem + 1.2vw, 1.75rem)` scales smoothly with the viewport between a minimum and a maximum, with no breakpoints. Keep a `rem` part in the middle value: pure `vw` font sizes don't grow when the user zooms, which fails WCAG 1.4.4.

→ *More:* §10.7 Fluid typography with `clamp()`, §14.6 Q37

Q165. How do you make images and videos responsive?

`img, video { max-width: 100%; height: auto; }` so they shrink to fit their container; `srcset` / `sizes` or `<picture>` so the browser downloads an appropriately sized file; `aspect-ratio` plus `object-fit: cover` for fixed-ratio frames; `image-set()` for CSS backgrounds; and for embeds (YouTube iframes) `width: 100%; aspect-ratio: 16 / 9`.

→ *More:* §10.8 Responsive images recap (and `image-set()`), §4.3 Responsive images — `srcset`, `sizes` and `<picture>`

Q166. What are user-preference media queries?

Media features that reflect the user's OS or browser settings: `prefers-color-scheme` (light/dark), `prefers-reduced-motion`, `prefers-contrast`, `forced-colors` (Windows high contrast), `prefers-reduced-transparency`. Honour them to respect users' needs.

→ *More:* §10.5 User-preference media queries

Q167. ⋆ How do you implement dark mode that respects the user's preference but allows a manual override?

Define semantic colour tokens (`--surface`, `--text`) and give them light and dark values — with `color-scheme: light dark` plus `light-dark(#fff, #111)`, or with `@media (prefers-color-scheme: dark)`. A toggle sets `data-theme="dark|light"` on `<html>` (which pins `color-scheme`) and saves the choice in `localStorage`; a tiny blocking script in `<head>` applies the saved theme before first paint to avoid a white flash.

Follow-up: Why not just invert colours with `filter: invert(1)`? It inverts images and brand colours too, can't respect contrast design, and costs paint performance. Design dark tokens deliberately (lighter, desaturated accents; surfaces not pure black).

→ *More:* §13.14 Theming and dark mode with custom properties, §10.5 User-preference media queries, §14.6 Q42

Q168. How do you test responsive design?

DevTools device mode at many widths (drag through every width, not just presets), real phones (touch, mobile Safari quirks), 200% and 400% zoom and the 320 CSS px reflow test (WCAG 1.4.10 — no horizontal scrolling), larger default font size, landscape orientation, and an automated check that `document.documentElement.scrollWidth` doesn't exceed the viewport.

→ *More:* §10.10 Testing responsive design

Q169. ⋆ My page has horizontal scrolling on mobile. How do you find and fix it?

Find the culprit: in DevTools run `[...document.querySelectorAll('body *')].filter(e => e.getBoundingClientRect().right > document.documentElement.clientWidth)` (lists the elements that stick out past the right edge) or add `* { outline: 1px solid red }`. Usual causes: fixed widths (`width: 600px`), `100vw` (which includes the scrollbar width), images without `max-width: 100%`, long URLs or words (`overflow-wrap: anywhere`), grid/flex items with `min-width: auto` (use `minmax(0, 1fr)` or `min-width: 0`), and negative margins. Don't hide it with `overflow-x: hidden` on `body` — that can break `position: sticky`.

→ *More:* §10.10 Testing responsive design, §9.10 Debugging, pitfalls and interview prep

Q170. What is the `aspect-ratio` property?

`aspect-ratio: 16 / 9` makes a box keep a width-to-height ratio as its width changes — for video embeds, thumbnails and avatars (`aspect-ratio: 1`). It replaced the padding-top percentage hack, and on images it reserves space to avoid layout shift.

→ *More:* §4.1 The `<img>` element, §11.9 `object-fit` and `object-position`

Centering techniques

Q171. ⋆ How do you center a `<div>` horizontally and vertically?

The two modern answers:

```
.parent { display: grid; place-items: center; min-height: 100svh; }
/* or */
.parent { display: flex; justify-content: center; align-items: center; }
```

The parent needs a height for vertical centering to be visible.

Follow-up: Why not `position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%)`? It works, but the element leaves the flow (the parent won't size around it) and can land on half pixels, blurring text. Grid or flex is simpler.

→ *More:* §8.7 Common patterns, §14.6 Q30

Q172. Name other ways to center something — and when each fits.

- Horizontally, a block with a width: `margin-inline: auto` (`margin: 0 auto`).
- Inline content or text: `text-align: center` on the parent.
- Positioned: `position: absolute; inset: 0; margin: auto;` with a set width and height (or `width: fit-content; height: fit-content` for unknown sizes), or `top: 50%; left: 50%; transform: translate(-50%, -50%)` (can blur on half pixels).
- One child in a flex container: `margin: auto` on the child.
- Vertically in a single line of text: `line-height` equal to the height (legacy).
- In normal block flow (no flex/grid): `align-content: center` on a block container works in all major browsers since 2024.

→ *More:* §7.5 Normal flow and formatting contexts, §8.5 Alignment, §9.6 Alignment

Q173. How do you center a modal that should scroll when its content is taller than the viewport?

Use `<dialog>` (which is centered by default) or a fixed overlay with grid centering, and cap the height: `max-height: calc(100svh - 2rem); overflow: auto;` on the dialog. With `align-items: center` and

overflowing content, the top can be cut off unreachable; `margin: auto` on the child (or `align-items: safe center`) avoids that, because auto margins never push content past the top edge.

→ *More:* §8.5 Alignment, §3.7 Keyboard and focus

Transitions, transforms, animations and visual effects

Q174. ⋆ Transitions vs animations — what's the difference and when do you use each?

A **transition** animates a property between two states when its value changes (on `:hover`, a class toggle): `transition: transform 200ms ease-out`. It needs a trigger and has only a start and end. A **keyframe animation** (`@keyframes` + `animation`) runs on its own, can loop, alternate, and have many intermediate steps. Use transitions for state changes in UI; animations for loaders, attention cues and multi-step sequences.

Follow-up: What events can you listen for? `transitionend`, `transitionstart`, `transitionrun`, `transitioncancel`, and `animationstart`, `animationiteration`, `animationend`, `animationcancel`.

→ *More:* §12.1 Transitions, §12.5 Keyframe animations

Q175. Transforms vs transitions — and how do they affect performance?

A **transform** changes how a box is drawn — `translate`, `rotate`, `scale`, `skew` — without affecting layout. A **transition** is *how* a change is animated over time. Transforms are cheap to animate because they skip layout and paint; transitioning layout properties (`width`, `top`, `margin`) forces layout on every frame.

→ *More:* §12.3 2D transforms, §12.7 Performance: animate `transform` and `opacity`

Q176. ⋆ Which properties are cheap to animate, and why?

`transform` and `opacity` (and `filter` in many cases). When the element is on its own compositor layer, the browser can change them in the **composite** step on the GPU (graphics processor), without recalculating layout or repainting, and even when the main thread is busy. Animating `width`, `height`, `top`, `left`, `margin` triggers layout + paint every frame; `background-color` and `box-shadow` trigger paint.

Follow-up: Why can `opacity` still be expensive? If the element isn't promoted to its own layer, changing it repaints; and a very large layer (full-screen blur) costs memory and GPU fill time.

→ *More:* §12.7 Performance: animate `transform` and `opacity`, §14.6 Q39

Q177. ⋆ My animation is janky. How do you optimise it?

Record it in DevTools' Performance panel and look for long layout/paint tasks and dropped frames. Animate only `transform`/`opacity` (e.g. `translate` instead of `left`, `scale` instead of `width`); use `will-change: transform` sparingly, just before the animation, to promote a layer; avoid layout thrashing in JS (reading `offsetHeight` right after writing styles in a loop); use `requestAnimationFrame` or the Web Animations API instead of `setInterval`; reduce the painted area and expensive effects like big blurs.

→ *More:* §12.7 Performance: animate `transform` and `opacity`

Q178. What does `will-change` do, and why not put it everywhere?

It hints that a property will change soon so the browser can prepare (usually promoting the element to its own GPU layer). Each layer costs memory, and too many layers slow everything down; it also creates a stacking context and a containing block. Add it shortly before an animation and remove it afterwards — or only on elements that animate constantly.

→ *More:* §12.7 Performance: animate `transform` and `opacity`

Q179. What are timing functions (easing)?

They control the speed curve of a transition or animation: `linear`, `ease` (default), `ease-in` (slow start — for things leaving), `ease-out` (slow end — for things entering, feels responsive), `ease-in-out`, `cubic-bezier(x1, y1, x2, y2)` for custom curves, `steps(n)` for frame-by-frame sprites, and `linear()` with many points to approximate springs and bounces.

→ *More:* §12.2 Timing functions (easing)

Q180. Does the order of transform functions matter?

Yes. Transforms apply right to left relative to the element's coordinate system, so `translateX(100px) rotate(45deg)` and `rotate(45deg) translateX(100px)` end in different places (the second moves along the rotated axis). The individual properties `translate`, `rotate`, `scale` are always applied in that fixed order and can be animated independently.

→ *More:* §12.3 2D transforms

Q181. How do you build a 3D flip card?

A container with `perspective: 1000px`; an inner element with `transform-style: preserve-3d` and `transition: transform .6s`; two faces stacked with `position: absolute` and `backface-visibility: hidden`, the back pre-rotated `rotateY(180deg)`; on hover or a class, rotate the inner element `rotateY(180deg)`. `overflow: hidden` or `opacity` / `filter` on the inner element flatten the 3D, so avoid them there.

→ *More:* §12.4 3D transforms

Q182. Why can't you transition from `display: none`, and how do you animate an element in and out today?

`display` is a discrete property: the element doesn't exist in the "from" state, so there's nothing to transition from. Modern CSS fixes this with `@starting-style { … }` (the style to transition *from* when the element first appears) plus `transition-behavior: allow-discrete` (keeps the element displayed until the exit transition finishes). Works well for `<dialog>`, `popover` and elements added by JS. Both are Baseline since August 2024, but the *exit* animation of `display` (and the `overlay` property that keeps a closing dialog in the top layer) is not in Firefox yet — there the element simply disappears without the fade, which is a harmless fallback.

→ *More:* §12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

Q183. What are view transitions and scroll-driven animations?

View transitions (`document.startViewTransition(() => updateDOM())`) animate between two DOM states — the browser snapshots old and new and cross-fades or morphs elements with the same `view-transition-name`. Same-document ones are Baseline since October 2025; cross-document (between pages, opted in with `@view-transition { navigation: auto }`) is Chrome and Safari only. **Scroll-driven animations** (`animation-timeline: scroll()` or `view()`) tie animation progress to scroll position instead of time — reading-progress bars, reveal-on-scroll — without JavaScript; not yet Baseline.

→ *More:* §12.10 View transitions, §12.11 Scroll-driven animations

Q184. CSS animations or JavaScript animations?

CSS (transitions/keyframes) for simple, declarative state changes — the browser can run `transform` / `opacity` off the main thread. The Web Animations API (`el.animate(keyframes, options)`) gives the same engine with JS control: pause, reverse, seek, promises when finished. Libraries (GSAP, Motion) for complex timelines, physics and SVG morphing. `requestAnimationFrame` for canvas or custom per-frame logic.

→ *More:* §12.6 The Web Animations API (WAAPI)

Q185. How would you implement a web design that uses non-standard fonts?

Self-host WOFF2 files with `@font-face`, set `font-display: swap` (show fallback text immediately, swap when the font arrives) or `optional` (use the font only if it arrives very fast), `preload` the one or two most important files, subset to the characters you need, consider a variable font instead of many weights, and tune the fallback with `size-adjust` / `ascent-override` to reduce layout shift. Always give a full fallback stack ending in a generic family. Two terms interviewers use: **FOIT** (flash of invisible text — text hidden while the font loads, the default for up to ~3 s) and **FOUT** (flash of unstyled text — fallback first, then a swap that can shift layout); `font-display` chooses between them.

→ *More:* §11.1 Fonts, §11.2 Loading fonts well: `font-display`, preload, subsetting, §14.6 Q38

Q186. How do `object-fit` and `object-position` work?

They control how a replaced element's content (an image or video) fills its box when the ratio differs: `cover` fills and crops, `contain` fits entirely with letterboxing, `fill` (default) stretches, `none` keeps natural size, `scale-down` picks the smaller of `none` and `contain`. `object-position` chooses which part stays visible (`top center`).

→ *More:* §11.9 `object-fit` and `object-position`

Q187. How do you make a gradient border, a glassmorphism card, or gradient text?

Gradient border: two backgrounds — `background: linear-gradient(#fff,#fff) padding-box, linear-gradient(90deg,#f0f,#0ff) border-box; border: 3px solid transparent;`. Glass: semi-transparent background + `backdrop-filter: blur(12px)` (check text contrast). Gradient text: `background: linear-gradient(…); background-clip: text; -webkit-background-clip: text; color: transparent;` (keep a fallback colour).

→ *More:* §11.5 Backgrounds and gradients, §11.8 Filters, blend modes, clipping and masks

---

Custom properties and modern CSS

Q188. How do CSS custom properties (variables) work?

Declared with two dashes — `:root { --brand: #1d4ed8; }` — and read with `var(--brand, fallback)`. They follow the cascade and **inherit**, so you can redefine them on any element or in a media query and all descendants update; JavaScript can change them at runtime (`el.style.setProperty('--x', '10px')`). An invalid value at use time makes the property behave as `unset` ("invalid at computed-value time"), not fall back to an earlier declaration.

Follow-up: How do you set a variable from JavaScript and read it back? `el.style.setProperty('--gap', '2rem')` and `getComputedStyle(el).getPropertyValue('--gap')`.

→ *More:* §6.9 Custom properties (CSS variables)

Q189. ⋆ What are the advantages of CSS variables over Sass variables?

Sass variables are replaced at **compile time** and disappear — they can't change per element, per media query (without duplication) or at runtime. CSS custom properties are **live** in the browser: they cascade, inherit, respond to media/container queries and JavaScript, and power theming (dark mode is just redefining tokens). Sass still helps with loops and mixins.

→ *More:* §6.9 Custom properties (CSS variables), §13.12 Preprocessors (Sass) and PostCSS, §14.6 Q19

Q190. What does `@property` add to custom properties?

`@property --angle { syntax: '<angle>'; inherits: false; initial-value: 0deg; }` registers a *typed* custom property. Because the browser now knows it's an angle (not just text), it can **animate** it — e.g. a rotating conic-gradient border — and validate it. Baseline since July 2024.

→ *More:* §13.14 Theming and dark mode with custom properties

Q191. ⋆ What is the `:has()` selector, and what problems does it solve?

`:has()` selects an element based on what it contains or what follows it — the long-wanted "parent selector": `.card:has(img)` (cards with an image), `form:has(:user-invalid) .submit` (style the button when any field is invalid), `h2:has(+ p)` (an h2 followed by a paragraph — a previous-sibling selector), `body:has(dialog[open])` (lock scrolling while a dialog is open). It removes a lot of JavaScript class-toggling. Baseline since December 2023.

Follow-up: Is `:has()` slow? Browsers optimise common cases well; avoid very broad anchors like `body:has(.x)` that must be re-checked on every DOM change in big documents, but normal use is fine.

→ *More:* §13.3 `:has()` — the relational ("parent") selector, §14.6 Q21

Q192. ⋆ `:is()` vs `:where()`?

Both take a selector list and match any of them, shortening long selectors (`:is(h1, h2, h3) a`). The difference is specificity: `:is()` takes its most specific argument's, `:where()` is always zero — use `:where()` in resets and library defaults so users can override them with a single class. Both are "forgiving": one invalid selector in the list doesn't kill the whole rule.

→ *More:* §13.2 `:is()` and `:where()`, §14.6 Q20

Q193. ⋆ What are cascade layers (`@layer`), and how do they help manage CSS at scale?

`@layer reset, vendor, base, components, utilities;` declares named layers in priority order (later wins). The cascade compares **layers before specificity**, so a simple `.btn` in `components` beats `#app .vendor .btn` in `vendor`. They end specificity wars with third-party CSS and make the order of your codebase explicit. Unlayered styles beat all layered ones; for `!important` the layer order reverses. Baseline since 2022.

Follow-up: What happens to styles not in any layer? They form an implicit final layer that beats all named layers for normal declarations — so wrapping third-party CSS in a low layer lets all your

unlayered styles win.

→ *More:* §13.4 Cascade layers (`@layer`), §14.6 Q22

Q194. Is CSS nesting native now? How does it differ from Sass nesting?

Yes — native nesting is Baseline since late 2023: `.card { & h2 { … } &:hover { … } }`. Differences from Sass: nested selectors behave like they're wrapped in `:is()` (so specificity follows `:is()` rules), and you can't concatenate strings to build class names (`&__title` works in Sass, not in CSS). Don't nest deeply — it increases specificity and couples CSS to the DOM.

→ *More:* §13.1 Native CSS nesting

Q195. What is `@scope`?

`@scope (.card) to (.card-content) { img { … } }` limits rules to a subtree, with an optional lower boundary ("donut scope") so styles don't leak into nested components. When two scoped rules conflict, the one whose scope root is **closer** wins (scope proximity). Baseline newly available since March 2026.

→ *More:* §13.5 `@scope`

Q196. ⋆ What are CSS logical properties, and why are they important?

Properties that use flow-relative directions instead of physical ones: `margin-inline-start` instead of `margin-left`, `padding-block` instead of top/bottom, `inset-inline-end`, `inline-size` instead of `width`. They adapt automatically to right-to-left languages and vertical writing modes, so one stylesheet works for Arabic and English.

→ *More:* §13.6 Logical properties

Q197. What is `@supports`?

A feature query: `@supports (display: grid) { … }` or `@supports selector(:has(a)) { … }` applies rules only if the browser supports the property-value pair or selector. It's the CSS tool for progressive enhancement; `not`, `and` and `or` combine conditions.

→ *More:* §10.6 Container queries

Q198. What is CSS anchor positioning?

A native way to tether one element to another — a tooltip or dropdown positioned next to its button without JavaScript measurement. The anchor gets `anchor-name: --btn`; the floating element gets `position: absolute; position-anchor: --btn; position-area: bottom;` (or `top: anchor(bottom)`), and `position-try-fallbacks: flip-block` flips it when there's no room. Its core (`anchor-name`, `anchor()`, `position-area`, `position-try-fallbacks`) became Baseline newly available in January 2026 when Firefox 147 shipped it; a few newer extras (such as some `position-visibility` values) are still Safari-only, so check before using them. It pairs well with the `popover` attribute.

CSS architecture: preprocessors, BEM, utility-first and CSS-in-JS

Q199. ⋆ What are the advantages and disadvantages of CSS preprocessors (Sass, Less)?

Pros: variables, nesting, mixins (reusable blocks with parameters), functions and loops, partials combined into one file, colour maths. Cons: a build step, compiled output can get bloated (deep nesting, `@extend`), and debugging needs source maps. Much of what made Sass essential is now native (custom properties, nesting, `color-mix()`, `@layer`); many teams now use **PostCSS** (a plugin pipeline — Autoprefixer, minification) and plain modern CSS.

→ *More:* §13.12 Preprocessors (Sass) and PostCSS

Q200. What is BEM?

A naming convention — **Block, Element, Modifier**: `.card` (block), `.card__title` (element of the block), `.card--featured` (variant). Every selector is a single flat class, so specificity stays low and equal, names say where styles belong, and components don't leak into each other. The downside is long class names and discipline needed across a team.

→ *More:* §13.7 Why CSS needs an architecture — and BEM

Q201. What is utility-first CSS (Tailwind)? Pros and cons?

You style with many small single-purpose classes in the HTML (`flex items-center gap-4 p-6 rounded-xl`). Tailwind v4 scans your source files and generates only the classes you use. Pros: no naming, no dead CSS, small final stylesheet, consistent design tokens, fast iteration. Cons: long class lists in markup, a learning curve, repeated class lists unless you extract components, and the HTML mixes structure and presentation. Responsive variants use prefixes (`md:grid-cols-3`), mobile-first.

Follow-up: How do you avoid repeating long class lists? Extract a component in your framework (a `Button` component), not `@apply` everywhere — Tailwind itself recommends components first.

→ *More:* §13.8 Utility-first CSS and Tailwind CSS v4

Q202. What are CSS Modules?

A build-time technique: each `.module.css` file's class names are rewritten to unique names (`.title` → `.Card_title__x7a2`) and imported into JavaScript as an object (`styles.title`). You get locally scoped class names with plain CSS and zero runtime cost.

→ *More:* §13.9 CSS Modules

Q203. What is CSS-in-JS? Pros and cons?

Writing styles in JavaScript, co-located with components (styled-components, Emotion, or zero-runtime tools like vanilla-extract, Linaria, StyleX). Pros: scoping, dynamic styles from props, dead-code elimination with the component. Cons of *runtime* libraries: they generate and inject CSS while rendering, adding JavaScript and slowing rendering, and they complicate server components and streaming SSR. The trend is toward zero-runtime or native CSS.

→ *More:* §13.10 CSS-in-JS

Q204. ⋆ BEM vs CSS Modules vs Tailwind vs CSS-in-JS — how do you choose?

All four solve CSS's **global namespace** problem in different ways: a convention (BEM), build-time unique names (CSS Modules), generated utilities (Tailwind), or styles owned by JS components (CSS-in-JS). Choose by team and stack: plain CSS + BEM + `@layer` for small sites or server-rendered pages; CSS Modules or Tailwind for React apps; zero-runtime CSS-in-JS if you need typed, themeable component libraries.

→ *More:* §13.11 Choosing an approach, §14.6 Q41

Q205. What existing CSS frameworks have you used? How would you improve them?

Talk about one honestly: Bootstrap (ready components, consistent but every site looks similar and you ship unused CSS), Tailwind (utilities), component libraries (MUI, Chakra, shadcn/ui). Improvements interviewers like to hear: ship design tokens as custom properties, use `@layer` so overrides are easy, use logical properties for RTL, tree-shake unused CSS, and meet WCAG out of the box.

→ *More:* §13.11 Choosing an approach

Q206. What are design tokens?

Named, platform-independent design decisions — colours, spacing, font sizes, radii, shadows, durations. Usually in three tiers: **primitive** (`--blue-600`), **semantic** (`--color-action`), **component** (`--button-bg`). Components use only semantic/component tokens, so theming or rebranding means changing the mapping, not the components.

→ *More:* §13.14 Theming and dark mode with custom properties

Q207. How do you organise CSS in a large codebase so it stays maintainable?

A layer order (`@layer reset, base, components, utilities`), low and flat specificity (single classes, `:where()` for defaults), component-scoped styles (BEM, CSS Modules or `@scope`), design tokens as custom properties, no IDs or `!important` for styling, stylelint in CI, and deleting dead CSS when components are removed. Make the cascade predictable rather than fighting it.

→ *More:* §13.4 Cascade layers (`@layer`), §13.7 Why CSS needs an architecture — and BEM

Performance and the browser rendering pipeline

Q208. ⋆ What happens when you type a URL and press Enter?

The browser checks caches, resolves the domain to an IP address (DNS), opens a TCP connection and a TLS handshake (HTTPS) — or QUIC for HTTP/3 — sends the HTTP request, and receives the response. It then parses HTML into the DOM, fetches CSS/JS/images, builds the CSSOM from the CSS, combines them into the render tree, computes layout, paints pixels and composites layers onto the screen; JavaScript runs along the way and can change things.

Follow-up: What would you say about caching at each step? Browser memory/disk cache and service worker before the network; DNS caches in the OS and resolver; HTTP caching headers (`Cache-Control`, `ETag`) decide whether the response is reused or revalidated.

→ *More:* §1.6 "What happens when you type a URL and press Enter?", §14.6 Q43

Q209. ⋆ Explain the browser rendering pipeline (critical rendering path).

1. **Parse HTML → DOM**. 2. **Parse CSS → CSSOM** (CSS Object Model). 3. **Style**: match rules to each element, computing styles (the visible elements form the **render tree**). 4. **Layout** (reflow): compute every box's size and position. 5. **Paint**: draw text, colours, borders and images into layers. 6. **Composite**: combine layers in the right order on the GPU. The **critical rendering path** is the minimum work needed for the first paint, and you speed it up by reducing render-blocking CSS/JS.

Follow-up: What does JavaScript do to this pipeline? A parser-blocking script stops DOM construction; JS that changes styles or the DOM re-runs style, layout and paint for the affected part.

→ *More:* §1.7 The browser rendering pipeline, §14.1 The critical rendering path

Q210. ⋆ What is the difference between layout (reflow), painting and compositing?

Layout calculates geometry — triggered by changes to size or position (`width`, `font-size`, adding elements); it is the most expensive because it can cascade through the page. **Paint** fills in pixels — triggered by visual changes that don't move anything (`color`, `background`, `box-shadow`). **Compositing** only reassembles already-painted layers — `transform`, `opacity` — and is the cheapest.

Follow-up: Give one property for each: `width` → layout, `color` → paint, `transform` → composite only.

→ *More:* §1.7 The browser rendering pipeline, §12.7 Performance: animate `transform` and `opacity`

Q211. ⋆ What causes reflow and repaint, and how do you reduce them?

Reflow: changing geometry, adding/removing DOM nodes, changing fonts or window size, and **reading** layout values (`offsetWidth`, `getBoundingClientRect()`) after a write, which forces a synchronous layout. Reduce it by batching DOM reads before writes, changing classes instead of many inline styles, animating `transform`/`opacity`, using `contain: layout paint` / `content-visibility: auto` to limit the affected area, and avoiding **layout thrashing** — alternating writes and reads (`el.style.width = …; h += el.offsetHeight`) in a loop, which forces a fresh layout on every iteration; read everything first, then write.

→ *More:* §1.7 The browser rendering pipeline, §12.7 Performance: animate `transform` and `opacity`

Q212. What are render-blocking resources, and how do you eliminate them?

CSS in the `<head>` blocks rendering (the browser won't paint until it has the CSSOM, to avoid FOUC). **Synchronous scripts** block HTML parsing (and wait for preceding CSS). Fixes: inline the critical CSS and load the rest without blocking; split CSS by media (`media="print"` doesn't block); `defer` or `async` scripts; remove unused CSS/JS; avoid `@import` chains.

→ *More:* §14.2 Render-blocking resources, §14.6 Q44

Q213. What is critical CSS?

The minimal CSS needed to render the above-the-fold (initially visible) content, inlined in a `<style>` in the `<head>` so first paint doesn't wait for a network request; the full stylesheet loads afterwards. Tools like Critters/Beasties extract it automatically. Trade-off: inlined CSS isn't cached across pages, so keep it small (≈ 14 KB, roughly what fits in the first round trip).

→ *More:* §14.2 Render-blocking resources

Q214. ⋆ What are the Core Web Vitals and their thresholds?

Google's user-experience metrics, measured at the 75th percentile of real visits: **LCP** (Largest Contentful Paint — when the biggest image or text block is drawn) ≤ 2.5 s; **INP** (Interaction to Next Paint — the delay from a click/tap/keypress to the next visual update, across the visit) ≤ 200 ms; **CLS** (Cumulative Layout Shift — how much visible content unexpectedly moves) ≤ 0.1. INP replaced FID in March 2024.

Follow-up: Why did INP replace FID? FID measured only the input delay of the first interaction; INP measures the full latency (delay + processing + rendering) of the slowest interactions across the whole visit, which matches what users feel.

→ *More:* §14.3 Core Web Vitals: LCP, INP, CLS, §14.6 Q45

Q215. How do you improve LCP, INP and CLS?

LCP: fast server response (TTFB) and CDN; make the LCP image discoverable in the HTML (not CSS or JS), `fetchpriority="high"`, never lazy-load it, compress it (AVIF/WebP, right size); remove render-blocking CSS/JS. **INP**: break up long JavaScript tasks (> 50 ms), yield to the main thread, do less work in event handlers, avoid huge DOMs. **CLS**: `width` / `height` or `aspect-ratio` on media, reserve space for ads and embeds, don't insert content above existing content, use `font-display` with metric-matched fallbacks, animate with `transform`.

→ *More:* §14.4 Fixing each metric (and the other basics)

Q216. Lab data vs field data?

Lab data (Lighthouse, DevTools) is measured in one simulated environment — reproducible, good for debugging, but it can't measure real INP. **Field data** (CrUX, the Chrome User Experience Report, or your own RUM — real-user monitoring with the `web-vitals` library) comes from actual visitors on their devices and networks; Google uses field data for ranking.

→ *More:* §14.3 Core Web Vitals: LCP, INP, CLS

Q217. ⋆ What are some "gotchas" for writing efficient CSS?

Big, unused stylesheets (download and parse cost); `@import` chains; expensive effects on large areas (big `box-shadow`, `filter: blur`, `backdrop-filter`) especially during scroll or animation; animating layout properties; very deep, overqualified selectors (`div.nav ul li a`) mainly hurt maintainability; universal selectors with costly properties; and huge numbers of DOM nodes. Use `content-visibility: auto` to skip rendering off-screen sections.

→ *More:* §14.4 Fixing each metric (and the other basics), §12.7 Performance: animate `transform` and `opacity`

Q218. ⋆ What are some ways to improve scrolling performance?

Use passive event listeners for `touchstart` / `wheel` (`{ passive: true }`); throttle scroll handlers or replace them with `IntersectionObserver` (lazy loading, reveal effects) and scroll-driven animations; avoid `background-attachment: fixed` and big repaints; promote animated elements with `transform`; use `content-visibility: auto` for long pages; virtualise very long lists (render only visible rows).

→ *More:* §12.7 Performance: animate `transform` and `opacity`, §14.4 Fixing each metric (and the other basics)

Q219. What tools would you use to find a performance problem?

Chrome DevTools: **Performance** panel (flame chart of scripting, style, layout, paint; long tasks), **Network** panel (waterfall, blocking requests, sizes), **Lighthouse** (lab audit with suggestions), the **Rendering** tab (paint flashing, layout-shift regions, layer borders), **Coverage** (unused CSS/JS); WebPageTest for real devices and networks; and field data from CrUX / PageSpeed Insights or RUM.

→ *More:* §1.8 Browser DevTools, §14.3 Core Web Vitals: LCP, INP, CLS

Q220. How do you optimise asset loading on a page overall?

Fewer and smaller bytes: minify and compress (Brotli/gzip), modern image formats with the right sizes, subset WOFF2 fonts, remove unused CSS/JS, code-split JS. Earlier: CDN, HTTP/2 or 3, `preconnect` / `preload` for critical resources, critical CSS. Smarter: `defer` / `async`, lazy-load below-the-fold images and iframes, long `Cache-Control` lifetimes with fingerprinted (hashed) file names.

→ *More:* §14.4 Fixing each metric (and the other basics), §1.5 HTTP basics — requests and responses

---

"Build this layout" — common live-coding tasks

Interviewers often hand you a screenshot or a sentence and watch you build it in 10–30 minutes. Say what element you choose and why, start with semantic HTML, then layout, then polish (focus, responsive,

reduced motion).

Q221. ⋆ Build the holy grail layout: header, footer, a fluid main column and two sidebars; footer at the bottom even when content is short.

```
body { min-height: 100svh; margin: 0; display: grid;
  grid-template: "head head head" auto "nav main ads" 1fr "foot foot foot" auto
                 / 12rem minmax(0, 1fr) 12rem; }
header { grid-area: head } nav { grid-area: nav } main { grid-area: main }
aside { grid-area: ads } footer { grid-area: foot }
@media (width < 48rem) {
  body { grid-template: "head" auto "nav" auto "main" 1fr "ads" auto "foot" auto / 1fr; }
}
```

The `1fr` row (the main row) absorbs extra height, which keeps the footer at the bottom. In `grid-template` each row size follows its own string, so in the one-column version `main` must carry the `1fr` — writing `"foot" 1fr` would make the *footer* row swallow all the spare height.

Follow-up: Why `minmax(0, 1fr)` for the main column? So long content (code blocks, tables) can't force the column wider than the space and cause horizontal scrolling.

→ *More:* §9.9 Patterns

Q222. ⋆ Build a sticky footer (footer at the bottom of the viewport on short pages, after the content on long ones).

```
body { min-height: 100svh; display: flex; flex-direction: column; margin: 0; }
main { flex: 1; }
```

Or grid: `body { display: grid; grid-template-rows: auto 1fr auto; min-height: 100svh; }`. Don't use `position: fixed` — that covers content on long pages.

→ *More:* §8.7 Common patterns

Q223. ⋆ Build a responsive navbar: logo on the left, links on the right; on mobile, a menu button toggles the links.

HTML: `<header><a class="logo" href="/">Acme</a><nav aria-label="Main"><button aria-expanded="false" aria-controls="menu">Menu</button><ul id="menu">…</ul></nav></header>`.

```
header { display: flex; align-items: center; justify-content: space-between; gap: 1rem; }
#menu { display: flex; gap: 1rem; list-style: none; }
nav button { display: none; }
@media (width < 40rem) {
  nav button { display: block; }
  #menu { display: none; flex-direction: column; }
  nav:has(button[aria-expanded="true"]) #menu { display: flex; }
}
```

A few lines of JS flip `aria-expanded`.

Follow-up: Why not a CSS-only checkbox hack for the menu? It works without JS but announces as a checkbox, not a menu button, and can't manage `aria-expanded`; a real `<button>` with a few lines of JS is more accessible.

→ *More:* §10.4 Mobile-first in practice, §8.7 Common patterns

Q224. Build a responsive card grid: 1 column on phones, as many as fit on wider screens, equal-height cards with the button at the bottom.

```
.grid { display: grid; gap: 1.5rem;
  grid-template-columns: repeat(auto-fill, minmax(min(18rem, 100%), 1fr)); }
.card { display: flex; flex-direction: column; }
.card .actions { margin-top: auto; }
.card img { width: 100%; aspect-ratio: 16 / 9; object-fit: cover; }
```

Grid stretches the cards in each row to the same height; `margin-top: auto` pushes the button down. Use `subgrid` if titles must align across cards.

→ *More:* §9.9 Patterns, §9.7 Subgrid

Q225. Build a two-column layout with a fixed-width sidebar and a fluid content area (sidebar stacks on mobile).

```
.layout { display: grid; grid-template-columns: 16rem minmax(0, 1fr); gap: 2rem; }
@media (width < 48rem) { .layout { grid-template-columns: 1fr; } }
```

A breakpoint-free alternative ("sidebar" pattern): `display: flex; flex-wrap: wrap;` with the sidebar `flex: 1 1 16rem` and the content `flex: 999 1 30rem` — the content wraps under when it would get narrower than 30rem.

→ *More:* §10.9 Intrinsic layouts — responsive without breakpoints

Q226. ⋆ Build an accessible modal dialog.

Use `<dialog>` with `showModal()`: focus moves in, the rest of the page is inert, Escape closes, and it's in the top layer. Add a heading referenced by `aria-labelledby`, a close button, `::backdrop` styling, and return focus to the trigger on close (the browser does this for `<dialog>`). Lock background scroll with `body:has(dialog[open]) { overflow: hidden; }`.

```
dialog { max-width: min(32rem, 100% - 2rem); max-height: calc(100svh - 2rem); overflow: auto; }
dialog::backdrop { background: rgb(0 0 0 / .5); }
```

Follow-up: What if you can't use `<dialog>`? Render the modal at the end of `<body>`, set `inert` on the rest of the page, move focus in, handle Escape, and restore focus on close — exactly what `<dialog>` does for you.

→ *More:* §3.7 Keyboard and focus, §12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

Q227. Build a tooltip with CSS only.

```
.tip { position: relative; }
.tip [role="tooltip"] { position: absolute; bottom: calc(100% + 8px); left: 50%;
  translate: -50% 0; white-space: nowrap; opacity: 0; pointer-events: none;
  transition: opacity .15s; }
.tip:is(:hover, :focus-within) [role="tooltip"] { opacity: 1; }
```

Make it show on keyboard focus too (`:focus-within`), link it with `aria-describedby`, keep it hoverable and dismissable with Escape (WCAG 1.4.13). Mention that anchor positioning plus `popover` now handles flipping at screen edges natively.

→ *More:* §7.6 Positioning

Q228. Build a dropdown menu.

A `<button aria-expanded aria-controls>` toggles a positioned list (`position: absolute; top: 100%` inside a `position: relative` wrapper), with links inside. Close on Escape and on outside click, return focus to the button. Native options: `<details><summary>`, or the `popover` attribute (`<button popovertarget="m">` + `<div id="m" popover>`), which handles light-dismiss and the top layer without JS.

→ *More:* §3.6 ARIA, §12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

Q229. Build tabs or an accordion with minimal JavaScript.

Accordion: native `<details name="faq"><summary>Question</summary>Answer</details>` — giving several `<details>` the same `name` makes them exclusive (only one open). Tabs: `role="tablist"` of `<button role="tab" aria-selected aria-controls>`, panels with `role="tabpanel"` and `hidden` on inactive ones; arrow keys move between tabs, Tab moves into the panel.

→ *More:* §3.6 ARIA

Q230. Build a CSS-only toggle switch from a checkbox.

```css
.switch input { appearance: none; inline-size: 3rem; block-size: 1.6rem;
  border-radius: 1rem; background: #999; position: relative; cursor: pointer; }
.switch input::before { content: ""; position: absolute; inset: .2rem auto .2rem .2rem;
  aspect-ratio: 1; border-radius: 50%; background: #fff; transition: translate .2s; }
.switch input:checked { background: #1d4ed8; }
.switch input:checked::before { translate: 1.4rem 0; }
```

Keep the real `<input type="checkbox" role="switch">` and a `<label>` so it stays keyboard- and screen-reader-accessible; keep `:focus-visible` styles.

→ *More:* §6.3 Pseudo-classes and pseudo-elements, §5.3 Input types

Q231. Build a loading spinner and a skeleton loader.

```css
.spinner { inline-size: 2rem; aspect-ratio: 1; border-radius: 50%;
  border: 4px solid #ddd; border-top-color: #1d4ed8; animation: spin 1s linear infinite; }
@keyframes spin { to { rotate: 1turn; } }
.skeleton { background: linear-gradient(90deg, #eee 25%, #f5f5f5 50%, #eee 75%) 0 0 / 200% 100%;
  animation: shimmer 1.5s infinite; }
@keyframes shimmer { to { background-position: -200% 0; } }
```

Add `role="status"` with visually hidden "Loading…" text, and slow or stop the motion under `prefers-reduced-motion: reduce`.

→ *More:* §12.5 Keyframe animations, §12.9 Reduced motion and accessibility

Q232. Make a full-bleed hero section with text over an image, readable at all sizes.

```css
.hero { display: grid; min-height: 70svh; }
.hero > * { grid-area: 1 / 1; }              /* stack image and text in one cell */
.hero img { width: 100%; height: 100%; object-fit: cover; }
.hero .text { align-self: end; padding: 2rem;
  background: linear-gradient(transparent, rgb(0 0 0 / .7)); color: #fff; }
```

Grid stacking avoids absolute positioning; the dark gradient helps keep text contrast at 4.5:1 or better over any photo (check the lightest part of the image). The hero image is the likely LCP element: give it `fetchpriority="high"` and don't lazy-load it.

→ *More:* §9.4 Placing items, §11.9 `object-fit` and `object-position`

Q233. Build a responsive data table that works on small screens.

Keep a real `<table>` with `<caption>` and `<th scope>`. Wrap it in a container with `overflow-x: auto` (plus `tabindex="0"`, `role="region"` and an `aria-label` so keyboard users can scroll it), and optionally make the first column `position: sticky; left: 0`. Turning rows into stacked cards with `display: block` on cells can remove table semantics in some browsers — test with a screen reader.

→ *More:* §5.1 Table semantics, §10.4 Mobile-first in practice

Q234. Style a custom checkbox or radio button accessibly.

Keep the native input (so keyboard, focus and screen readers work), remove its default look with `appearance: none`, size it, draw the checked state with `::before` or a background, and use `:checked`, `:focus-visible`, `:disabled` for states. The simplest option: `accent-color: #1d4ed8` restyles native checkboxes, radios, range sliders and progress bars in one line.

→ *More:* §5.3 Input types, §6.3 Pseudo-classes and pseudo-elements

---

Scenarios, troubleshooting and senior-level questions

Q235. A colleague's CSS isn't applying. How do you debug it?

Open DevTools → Elements → Styles on the element: is the rule listed at all (if not: selector doesn't match, file not loaded — check Network, or a syntax error above it)? Is it **crossed out** (overridden — the Computed tab shows which rule wins and why: specificity, layer, order, `!important`)? Is there a warning icon (invalid value or a property that doesn't apply, like `width` on an inline element or `z-index` on a static one)? Also check media/container query conditions and inherited vs initial values.

→ *More:* §1.8 Browser DevTools, §6.10 How it all fits together

Q236. Why doesn't `height: 100%` work on my element?

Percentage heights resolve against the parent's **height**, and if the parent's height depends on its content (`auto`), the percentage is treated as `auto`. Give every ancestor a definite height, or better: use `min-height: 100svh` on the page container, flex/grid stretching (`align-items: stretch` is the default), or `height: 100%` inside a grid/flex item, which does have a definite size.

→ *More:* §7.1 The box model

Q237. My sticky navbar works on desktop but not on mobile Safari (or not at all). What could be wrong?

Check the usual sticky killers: an ancestor with `overflow: hidden/auto` (often added on `body` or a wrapper to stop horizontal scrolling — `overflow-x: clip` is the safe alternative), missing `top: 0`, a parent no taller than the nav, `align-self: stretch` inside flex/grid. Older Safari needed `-webkit-sticky`. Also check the `100vh`/toolbar behaviour if the nav sits inside a full-height container.

→ *More:* §7.6 Positioning, §7.7 Overflow

Q238. ⋆ The text in my flex row overflows instead of wrapping (or my grid breaks on mobile with horizontal scrolling). What's wrong?

Automatic minimum sizes: flex items have `min-width: auto` and `1fr` grid tracks have an `auto` minimum, so long words, URLs, `<pre>` blocks, tables or wide images refuse to shrink. Fix with `min-width: 0` on the flex item, `minmax(0, 1fr)` for grid tracks, `overflow-wrap: anywhere` for text, and `max-width: 100%` for media. Also check for fixed pixel widths and `minmax(300px, 1fr)` on screens narrower than 300px (use `min(300px, 100%)`).

→ *More:* §8.4 The `flex` shorthand and minimum sizes, §9.2 Defining tracks

Q239. An image inside a link adds strange extra height, or a button and an input next to each other don't line up. Why?

Inline-level boxes sit on the text baseline and include the line's descender space and the parent's `line-height`. Set `display: block` on the image, or align form controls with flexbox (`align-items: center`) and `font: inherit` so the input and button share font metrics.

→ *More:* §7.4 `display`

Q240. A page shows a flash of unstyled content (FOUC) or the dark theme flashes white first. How do you fix it?

FOUC: stylesheets loaded late (by JavaScript, at the end of the body, or via `@import` chains) — put `<link rel="stylesheet">` in the head, or inline critical CSS. Dark-mode flash: the theme is applied after first paint — run a tiny inline script in `<head>` that reads the saved theme and sets `data-theme` before the body renders, and set `color-scheme` so the browser's default background is dark too.

→ *More:* §14.2 Render-blocking resources, §13.14 Theming and dark mode with custom properties

Q241. How do you make a site work well in Windows high-contrast (forced-colors) mode?

In forced-colours mode the OS replaces your colours with a small user palette, and removes backgrounds and box-shadows. Use real borders or outlines (transparent ones become visible) for focus rings and button edges, use SVG icons with `fill: currentColor`, don't convey state by background colour alone, and test with `@media (forced-colors: active)` (using system colours like `CanvasText`, `Highlight` if you need fixes).

→ *More:* §11.6 Borders, radius and outlines, §10.5 User-preference media queries

Q242. How would you make an existing page accessible — where do you start?

Run an automated audit (axe/Lighthouse) for quick wins, then a keyboard-only pass (focus visible, order logical, no traps, everything operable), then fix semantics: landmarks, one `<h1>` and a heading hierarchy, real buttons and links, labels on all inputs, alt text, sufficient contrast, `lang`, and error messages tied to fields. Then a screen-reader pass and zoom/reflow testing. Prioritise blockers (can't complete a key task) over cosmetic issues.

→ *More:* §3.10 Testing tools, §3.9 WCAG 2.2

Q243. How do you reduce CSS bundle size in a large app?

Measure with the DevTools Coverage tab; remove dead CSS (PurgeCSS, Tailwind's source scanning, CSS Modules tree-shaken with components); split CSS per route or component; replace heavy frameworks with what you use; minify with Lightning CSS or cssnano; prefer modern CSS (fewer prefixes and hacks) with a Browserslist target; and inline only the critical part.

→ *More:* §13.8 Utility-first CSS and Tailwind CSS v4, §14.4 Fixing each metric (and the other basics)

Q244. How would you set up theming for multiple brands in one component library?

Components consume only semantic custom properties (`--color-action`, `--radius-control`). Each brand is a set of token values applied with a class or `data-brand` attribute on a root element (`[data-brand="acme"] { --color-action: oklch(55% .2 260); }`), shipped in their own `@layer tokens`. Dark mode is another axis of the same tokens (`light-dark()`). Generate token files from one source (e.g. Style Dictionary) for web, iOS and Android.

→ *More:* §13.14 Theming and dark mode with custom properties

Q245. How do the HTML, CSS and JavaScript of a page work together, and what is the DOM vs the CSSOM?

HTML gives structure and content (parsed into the **DOM**, a tree of node objects JavaScript can read and change). CSS gives presentation (parsed into the **CSSOM**, the tree of style rules). The browser combines DOM and CSSOM to render. JavaScript adds behaviour: it can modify both trees, which triggers style, layout and paint again. Scripts can also block parsing, and a script that reads styles must wait for pending CSS — which is why loading order matters.

→ *More:* §1.7 The browser rendering pipeline, §14.1 The critical rendering path

Q246. How do you safely insert user content into the page?

Use `textContent` (or framework templating that escapes by default), never `innerHTML` with untrusted strings — that allows XSS. If you must render user HTML, sanitise it (DOMPurify, or the newer `setHTML()` Sanitizer API where supported). Add a Content Security Policy (CSP, an HTTP header restricting where scripts may load from) as a second line of defence.

Q247. What is clickjacking, and how do you prevent it?

An attacker loads your site in an invisible `<iframe>` over a decoy page, tricking users into clicking your buttons. Prevent it by forbidding framing: `Content-Security-Policy: frame-ancestors 'self'` (modern) or `X-Frame-Options: DENY` (legacy).

→ *More:* §4.7 Embedding other pages — `<iframe>`

Q248. What's your process for turning a design (Figma) into HTML and CSS?

Identify the content structure first and write semantic HTML; extract design tokens (colours, type scale, spacing) into custom properties; identify reusable components; choose layout per region (grid for the page, flex for components); build mobile-first with fluid sizing (`clamp`, `min()`), adding breakpoints only where the content needs them; then states (hover, focus, disabled, error, empty, loading), accessibility checks and cross-browser testing.

→ *More:* §10.4 Mobile-first in practice, §13.14 Theming and dark mode with custom properties

Q249. ⋆ What's new in CSS that you're excited about (or have used recently)?

Pick two or three you can explain: container queries (components that adapt to their space), `:has()` (parent selection, less JS), cascade layers (no more specificity wars), native nesting, `@scope`, subgrid, `@starting-style` and view transitions (smooth UI without libraries), anchor positioning with `popover` (tooltips without JS), `oklch()` / `color-mix()` / `light-dark()` for theming, and scroll-driven animations.

→ *More:* §13.3 `:has()` — the relational ("parent") selector, §13.4 Cascade layers (`@layer`), §10.6 Container queries

Q250. What is the `popover` attribute?

`<div id="menu" popover>` plus `<button popovertarget="menu">` gives a toggleable panel with no JavaScript: it renders in the top layer (above every `z-index`), closes on Escape or an outside click ("light dismiss"), and the browser handles focus return. `:popover-open` styles the open state. Baseline since January 2025.

→ *More:* §12.8 Enter and exit animations: `@starting-style` and `allow-discrete`, §7.8 `z-index` and stacking contexts

Sources of the question lists

Question wording and frequency only; all answers above are written in my own words.

- h5bp — *Front-end Developer Interview Questions*: HTML Questions, CSS Questions, Performance Questions, General Questions (github.com/h5bp/Front-end-Developer-Interview-Questions)
- GreatFrontEnd — *CSS Interview Questions Guide for 2026* and the HTML/CSS quiz question list (greatfrontend.com)
- GreatFrontEnd — *Machine Coding Interview Questions for Freshers (2026 Guide)* (for the "build this" tasks)
- InterviewBit — *HTML Interview Questions* (HTML, HTML5, Advanced HTML and HTML/CSS/JavaScript sections)
- golinuxcloud — *CSS Interview Questions and Answers*; vskills — *Top 100 CSS Interview Questions*
- DEV Community — *Most frequently asked front-end interview questions*
- web.dev — *New to the web platform* monthly posts (to check browser support for anchor positioning, popover, view transitions)
- This book's own Chapter 14 §14.6 (all 47 questions are included above and linked)